Kubernetes 入門

aoki-taquan

2026 年 03 月 15 日

目次

1 はじめに 11
1.1 Kubernetes とは 11
1.2 なぜ Kubernetes が必要なのか 11
1.3 Kubernetes の主な特徴 11
1.3.1 自動修復（Self-healing） 11
1.3.2 水平スケーリング 11
1.3.3 サービスディスカバリとロードバランシング 11
1.3.4 自動ロールアウトとロールバック 11
1.3.5 宣言的設定 12
1.4 このガイドについて 12
1.5 対象読者 12
1.6 前提知識 12
2 環境構築 13
2.1 ローカル開発環境の選択肢 13
2.2 kubectl のインストール 13
2.2.1 macOS 13
2.2.2 Linux 13
2.2.3 Windows 13
2.2.4 インストール確認 13
2.3 Minikube 14
2.3.1 インストール 14
2.3.2 クラスタの起動 14
2.3.3 クラスタの状態確認 14
2.3.4 管理コマンド 14
2.3.5 ダッシュボード 15
2.4 kind（Kubernetes IN Docker） 15
2.4.1 インストール 15
2.4.2 クラスタの作成 15
2.4.3 マルチノードクラスタ 15
2.4.4 ローカルイメージの読み込み 16
2.4.5 管理コマンド 16
2.5 k3d（k3s in Docker） 16
2.5.1 インストール 16
2.5.2 クラスタの作成 16
2.5.3 ローカルレジストリとの連携 17
2.5.4 管理コマンド 17
2.6 k3s 17
2.6.1 インストール 17
2.6.2 k3s の特徴 18

2.6.3 kubectl の使用 18
2.7 MicroK8s 18
2.7.1 インストール 18
2.7.2 アドオンの有効化 18
2.7.3 管理コマンド 19
2.8 Docker Desktop 19
2.8.1 有効化の手順 19
2.8.2 コンテキストの切り替え 19
2.9 ツールの選び方 19
2.10 kubectl の基本操作 20
2.10.1 ノードの確認 20
2.10.2 全リソースの確認 20
2.10.3 名前空間の確認 20
2.10.4 コンテキストの管理 20
3 基本概念 21
3.1 Kubernetes のアーキテクチャ 21
3.1.1 コントロールプレーン 21
3.1.2 ワーカーノード 21
3.2 主要なリソース 21
3.2.1 ワークロード系リソース 21
3.2.2 ネットワーク系リソース 21
3.2.3 設定・ストレージ系リソース 22
3.2.4 クラスタ管理系リソース 22
3.3 マニフェストファイル 22
3.3.1 apiVersion 22
3.4 ラベルとセレクタ 22
3.4.1 ラベル 22
3.4.2 セレクタ 23
3.5 kubectl の基本コマンド 23
3.5.1 出力フォーマット 23
4 Pod 24
4.1 Pod とは 24
4.2 Pod の作成 24
4.2.1 マニフェストで作成 24
4.2.2 コマンドで直接作成 24
4.3 Pod の状態確認 25
4.3.1 一覧表示 25
4.3.2 詳細情報 25
4.4 Pod のライフサイクル 25
4.5 Pod 内でのコマンド実行 25
4.6 Pod のログ確認 26
4.7 ポートフォワーディング 26

4.8 マルチコンテナ Pod 26
4.9 リソース制限 26
4.10 ヘルスチェック 27
4.10.1 livenessProbe 27
4.10.2 readinessProbe 27
4.10.3 startupProbe 27
4.11 Pod の削除 28
5 Deployment 29
5.1 Deployment とは 29
5.2 Deployment と ReplicaSet の関係 29
5.3 Deployment の作成 29
5.4 状態確認 30
5.5 スケーリング 30
5.5.1 コマンドでスケーリング 30
5.5.2 マニフェストでスケーリング 31
5.6 ローリングアップデート 31
5.6.1 イメージの更新 31
5.6.2 アップデートの進行状況を確認 31
5.6.3 アップデート戦略の設定 31
5.7 ロールバック 32
5.7.1 ロールアウト履歴の確認 32
5.7.2 特定リビジョンの詳細を確認 32
5.7.3 前のバージョンに戻す 32
5.7.4 特定のリビジョンに戻す 32
5.8 一時停止と再開 32
5.9 Deployment の削除 32
6 Service 34
6.1 Service とは 34
6.2 Service の種類 34
6.3 ClusterIP Service 34
6.3.1 確認 35
6.3.2 クラスタ内からのアクセス 35
6.4 NodePort Service 35
6.4.1 Minikube でのアクセス 35
6.5 LoadBalancer Service 35
6.6 DNS によるサービスディスカバリ 36
6.7 複数ポートの公開 36
6.8 Headless Service 37
6.9 Service の確認と削除 37
7 ConfigMap と Secret 38
7.1 設定管理の必要性 38
7.2 ConfigMap 38

7.2.1 ConfigMap の作成 38
7.2.2 ConfigMap の確認 38
7.2.3 ConfigMap の利用方法 39
7.3 Secret 40
7.3.1 Secret の種類 40
7.3.2 Secret の作成 40
7.3.3 Secret の利用方法 41
7.4 Secret の注意点 41
8 ストレージ 43
8.1 コンテナのストレージの課題 43
8.2 Volume 43
8.2.1 emptyDir 43
8.2.2 hostPath 43
8.3 PersistentVolume（PV）と PersistentVolumeClaim（PVC） 44
8.3.1 PV の作成 44
8.3.2 アクセスモード 44
8.3.3 再利用ポリシー 44
8.3.4 PVC の作成 45
8.3.5 PVC を Pod で使用 45
8.4 StorageClass 45
8.5 ストレージの確認 46
9 Namespace 47
9.1 Namespace とは 47
9.2 デフォルトの Namespace 47
9.3 Namespace の操作 47
9.3.1 作成 47
9.3.2 一覧表示 47
9.3.3 特定の Namespace 内のリソースを表示 47
9.3.4 デフォルト Namespace の変更 48
9.4 Namespace にリソースを作成 48
9.5 ResourceQuota 48
9.5.1 ResourceQuota の確認 49
9.6 LimitRange 49
9.7 Namespace の削除 50
10 Ingress 51
10.1 Ingress とは 51
10.2 Ingress Controller 51
10.2.1 主な Ingress Controller 51
10.2.2 NGINX Ingress Controller のインストール 51
10.3 基本的な Ingress の定義 51
10.4 パスベースのルーティング 52
10.4.1 pathType 53

10.5 ホスト名ベースのルーティング 53
10.6 TLS 終端 53
10.6.1 TLS 用 Secret の作成 54
10.6.2 Ingress の設定 54
10.7 cert-manager による自動 TLS 証明書管理 54
10.7.1 cert-manager のインストール 54
10.7.2 ClusterIssuer の作成 54
10.7.3 Ingress でのアノテーション 55
10.8 NGINX Ingress のアノテーション 55
10.9 Ingress の確認 56
11 Helm 57
11.1 Helm とは 57
11.2 インストール 57
11.3 Chart リポジトリ 57
11.3.1 リポジトリの追加 57
11.3.2 リポジトリの管理 58
11.3.3 Chart の検索 58
11.4 Chart のインストール 58
11.4.1 基本的なインストール 58
11.4.2 インストール前の確認 58
11.4.3 設定値のカスタマイズ 58
11.4.4 利用可能な設定値を確認 59
11.5 リリースの管理 59
11.5.1 リリースの一覧 59
11.5.2 アップグレード 59
11.5.3 ロールバック 60
11.5.4 アンインストール 60
11.6 独自 Chart の作成 60
11.6.1 Chart のひな形を作成 60
11.6.2 Chart.yaml 61
11.6.3 テンプレートの記法 61
11.6.4 ローカル Chart のインストール 61
11.7 よく使われるコミュニティ Chart 62
12 Kustomize 63
12.1 Kustomize とは 63
12.2 Helm と Kustomize の比較 63
12.3 基本的なディレクトリ構造 63
12.4 ベースの作成 64
12.4.1 マニフェストファイル 64
12.4.2 kustomization.yaml 64
12.4.3 ベースの適用 65
12.5 オーバーレイの作成 65

12.5.1 開発環境 65
12.5.2 本番環境 65
12.5.3 オーバーレイの適用 66
12.6 主な機能 66
12.6.1 namePrefix / nameSuffix 66
12.6.2 namespace 66
12.6.3 commonLabels / commonAnnotations 66
12.6.4 images 66
12.6.5 ConfigMapGenerator / SecretGenerator 67
12.6.6 patchesStrategicMerge 67
12.7 kustomize コマンド 68
13 ArgoCD と GitOps 69
13.1 GitOps とは 69
13.1.1 GitOps の原則 69
13.1.2 従来の CI/CD と GitOps の違い 69
13.2 主な GitOps ツール 69
13.3 Argo CD とは 69
13.3.1 Argo CD の主な特徴 69
13.4 Argo CD のインストール 70
13.4.1 kubectl でインストール 70
13.4.2 Helm でインストール 70
13.5 Argo CD CLI のインストール 70
13.6 Web UI へのアクセス 70
13.6.1 初期パスワードの取得 71
13.7 Application の作成 71
13.7.1 CLI で作成 71
13.7.2 マニフェストで作成 71
13.7.3 Helm Chart をソースにする場合 72
13.7.4 Kustomize をソースにする場合 72
13.8 同期（Sync） 72
13.8.1 同期の状態 72
13.8.2 手動同期 73
13.8.3 自動同期の設定 73
13.9 Application の管理 73
13.10 ApplicationSet による複数環境管理 73
13.11 GitOps のベストプラクティス 74
13.12 Flux（補足） 74
14 モニタリング 76
14.1 Kubernetes におけるモニタリングの重要性 76
14.2 モニタリングスタック 76
14.3 Prometheus 76
14.3.1 Prometheus とは 76

14.3.2 kube-prometheus-stack のインストール 76
14.3.3 インストールされるコンポーネント 77
14.3.4 Prometheus UI へのアクセス 77
14.3.5 PromQL（Prometheus Query Language） 77
14.3.6 ServiceMonitor 77
14.4 Grafana 78
14.4.1 Grafana UI へのアクセス 78
14.4.2 初期認証情報 78
14.4.3 プリインストールされたダッシュボード 78
14.4.4 カスタムダッシュボードの作成 78
14.5 Alertmanager 79
14.5.1 アラートルールの定義 79
14.5.2 通知先の設定 79
14.6 Loki（ログ集約） 80
14.6.1 インストール 80
14.6.2 Grafana で Loki のログを確認 80
14.6.3 LogQL（ログクエリ） 80
14.7 metrics-server 81
15 実践運用 82
15.1 アプリケーションのデプロイ例 82
15.1.1 Deployment の作成 82
15.1.2 Service の作成 83
15.1.3 ConfigMap の作成 83
15.1.4 一括適用 83
15.2 トラブルシューティング 83
15.2.1 Pod が起動しない場合 83
15.2.2 よくある Pod のエラー 84
15.2.3 Service に接続できない場合 84
15.3 リソースの監視 84
15.3.1 リソース使用量の確認 84
15.4 ラベルを活用した運用 85
15.4.1 ラベルの追加・変更 85
15.4.2 ラベルによるフィルタリング 85
15.5 便利な kubectl コマンド 85
15.6 次のステップ 85
15.6.1 参考リソース 86
16 Kubernetes ディストリビューションとクラスタ構築 87
16.1 ディストリビューションの全体像 87
16.2 kubeadm 87
16.2.1 kubeadm とは 87
16.2.2 前提条件 87
16.2.3 コンテナランタイムのインストール 88

16.2.4 kubeadm、kubelet、kubectl のインストール 88
16.2.5 コントロールプレーンの初期化 88
16.2.6 CNI（ネットワークプラグイン）のインストール 88
16.2.7 ワーカーノードの追加 89
16.2.8 クラスタのアップグレード 89
16.2.9 kubeadm の特徴まとめ 90
16.3 Kubespray 90
16.3.1 Kubespray とは 90
16.3.2 特徴 90
16.3.3 基本的な使い方 90
16.3.4 インベントリのカスタマイズ 90
16.3.5 クラスタの構築 91
16.3.6 kubeadm との使い分け 91
16.4 Cluster API（CAPI） 91
16.4.1 Cluster API とは 91
16.4.2 コンセプト 91
16.4.3 対応インフラプロバイダー 92
16.4.4 clusterctl のインストール 92
16.4.5 Management Cluster の初期化 92
16.4.6 Workload Cluster の作成 92
16.4.7 Cluster API のユースケース 93
16.5 k3s（本番環境での使用） 93
16.5.1 k3s の本番向け構成 93
16.5.2 kubeadm との比較 94
16.6 k0s 94
16.6.1 k0s とは 94
16.6.2 k0s の特徴 94
16.6.3 k3s との違い 94
16.6.4 インストールと起動 95
16.6.5 k0sctl によるマルチノード構成 95
16.7 Talos Linux 96
16.7.1 Talos Linux とは 96
16.7.2 Talos の特徴 96
16.7.3 talosctl のインストール 96
16.7.4 クラスタの構成ファイル生成 96
16.7.5 ノードへの設定適用 97
16.7.6 クラスタのブートストラップ 97
16.7.7 設定のカスタマイズ 97
16.7.8 クラスタの管理 98
16.7.9 Talos のローカル開発環境 98
16.7.10 Talos Factory / Image Factory 98
16.7.11 Cluster API との連携 99

16.8 RKE2 / Rancher 99
16.8.1 RKE2 とは 99
16.8.2 RKE2 の特徴 99
16.8.3 インストール 99
16.8.4 RKE2 の設定 100
16.8.5 Rancher 100
16.8.6 k3s / RKE2 / kubeadm の比較 101
16.9 OpenShift 101
16.9.1 OpenShift とは 101
16.9.2 OpenShift のエディション 101
16.9.3 標準 Kubernetes との主な違い 101
16.9.4 oc コマンド 102
16.9.5 Route 102
16.9.6 Security Context Constraints（SCC） 103
16.9.7 ローカル開発環境（OKD / CRC） 103
16.10 VMware Tanzu Kubernetes Grid 103
16.10.1 Tanzu とは 103
16.10.2 Tanzu の主な製品 104
16.10.3 特徴 104
16.11 コンテナ最適化 OS 104
16.11.1 Bottlerocket 104
16.11.2 Flatcar Container Linux 104
16.11.3 コンテナ最適化 OS の比較 105
16.12 マネージド Kubernetes サービス 105
16.12.1 CLI ツール 106
16.13 ディストリビューションの選び方 106

1 はじめに

1.1 Kubernetes とは

Kubernetes（クバネティス、略称: K8s）は、コンテナ化されたアプリケーションのデプロイ、スケーリング、管理を自動化するためのオープンソースプラットフォームです。Google が社内で使用していたコンテナ管理システム「Borg」の経験をもとに開発され、2014 年にオープンソースとして公開されました。現在は Cloud Native Computing Foundation（CNCF）によって管理されています。

Kubernetes という名前はギリシャ語で「操舵手」や「パイロット」を意味します。K8s という略称は、「K」と「s」の間に 8 文字あることに由来しています。

1.2 なぜ Kubernetes が必要なのか

コンテナ技術（Docker 等）の普及により、アプリケーションのパッケージングと実行が容易になりました。しかし、本番環境で多数のコンテナを運用するには、以下のような課題があります。

- **スケーリング**: トラフィックの増減に応じてコンテナ数を調整する必要がある
- **障害対応**: コンテナやノードの障害時に自動的に復旧させたい
- **ロードバランシング**: 複数のコンテナにトラフィックを分散させたい
- **ローリングアップデート**: ダウンタイムなしでアプリケーションを更新したい
- **設定管理**: 環境ごとに異なる設定を安全に管理したい

Kubernetes はこれらの課題を解決し、コンテナのオーケストレーション（統合管理）を実現します。

1.3 Kubernetes の主な特徴

1.3.1 自動修復（Self-healing）

コンテナが異常終了した場合、Kubernetes は自動的にコンテナを再起動します。ノードに障害が発生した場合は、別のノードでコンテナを再スケジュールします。

1.3.2 水平スケーリング

CPU 使用率やカスタムメトリクスに基づいて、アプリケーションのレプリカ数を自動的に増減できます。

1.3.3 サービスディスカバリとロードバランシング

Kubernetes は DNS 名や IP アドレスを使ってコンテナを公開できます。トラフィックが多い場合は自動的に負荷分散を行います。

1.3.4 自動ロールアウトとロールバック

アプリケーションの更新時に段階的に新しいバージョンをデプロイし、問題が発生した場合は自動的にロールバックできます。

1.3.5 宣言的設定

YAML や JSON のマニフェストファイルで望ましい状態を宣言し、Kubernetes がその状態を自動的に維持します。

1.4 このガイドについて

このガイドでは、Kubernetes の基本概念から実践的な使い方まで、初心者向けにステップバイステップで解説します。

1.5 対象読者

- コンテナ（Docker）の基本的な知識がある方
- Kubernetes をこれから学び始めたい方
- Linux コマンドの基本操作ができる方

1.6 前提知識

以下の知識があると、本ガイドの内容をより理解しやすくなります。

- Docker の基本操作（イメージのビルド、コンテナの起動・停止）
- YAML 記法の基礎
- Linux/macOS のターミナル操作
- ネットワークの基礎知識（IP アドレス、ポート、DNS）

2 環境構築

2.1 ローカル開発環境の選択肢

Kubernetes をローカル環境で試すためのツールがいくつかあります。

ツール	特徴	リソース消費	推奨用途
Minikube	最も一般的、豊富なアドオン	中	学習・開発
kind	Docker 上でマルチノード構成が可能	小	CI/CD・テスト
k3d	k3s を Docker 上で実行、高速起動	小	軽量開発・CI
k3s	軽量 K8s ディストリビューション	極小	エッジ・IoT・本番
MicroK8s	Snap 経由で簡単インストール	小	Ubuntu 環境
Docker Desktop	GUI 付き、ワンクリック有効化	大	初心者向け

2.2 kubectl のインストール

kubectl は Kubernetes クラスタを操作するためのコマンドラインツールです。どのローカル環境ツールを使う場合でも必要です。

2.2.1 macOS

```
brew install kubectl
```

2.2.2 Linux

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/
bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
rm kubectl
```

2.2.3 Windows

```
winget install Kubernetes.kubectl
```

2.2.4 インストール確認

```
kubectl version --client
```

2.3 Minikube

Minikube は最も広く使われているローカル Kubernetes 環境です。豊富なアドオン（dashboard、metrics-server、ingress など）を簡単に有効化できます。

2.3.1 インストール

```
# macOS
brew install minikube

# Linux
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
rm minikube-linux-amd64

# Windows
winget install Kubernetes.minikube
```

2.3.2 クラスタの起動

```
minikube start
```

リソースをカスタマイズする場合は以下のように指定します。

```
minikube start --cpus=4 --memory=8192 --driver=docker
```

2.3.3 クラスタの状態確認

```
minikube status
```

以下のような出力が表示されれば正常です。

```
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

2.3.4 管理コマンド

コマンド	説明
`minikube start`	クラスタを起動する
`minikube stop`	クラスタを停止する（データは保持）
`minikube delete`	クラスタを削除する

コマンド	説明
`minikube dashboard`	Web UI ダッシュボードを開く
`minikube ssh`	Minikube ノードに ssh 接続する
`minikube tunnel`	LoadBalancer サービスにアクセス可能にする
`minikube addons list`	利用可能なアドオンを一覧表示する
`minikube addons enable <名前>`	アドオンを有効にする

2.3.5 ダッシュボード

```
minikube dashboard
```

ダッシュボードではクラスタの状態を GUI で確認・管理できます。

2.4 kind（Kubernetes IN Docker）

kind は Docker コンテナをノードとして使用するツールです。マルチノードクラスタの構築が容易で、CI/CD 環境に適しています。

2.4.1 インストール

```
# macOS / Linux
brew install kind

# Go がインストール済みの場合
go install sigs.k8s.io/kind@latest
```

2.4.2 クラスタの作成

```
# シンプルなクラスタ作成
kind create cluster

# 名前を指定して作成
kind create cluster --name my-cluster
```

2.4.3 マルチノードクラスタ

設定ファイルを使ってマルチノード構成を作成できます。

```
# kind-config.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
```

```
  - role: worker
  - role: worker
```

```
kind create cluster --config kind-config.yaml
```

2.4.4 ローカルイメージの読み込み

kind ではローカルにビルドした Docker イメージを直接クラスタに読み込めます。

```
docker build -t my-app:latest .
kind load docker-image my-app:latest
```

2.4.5 管理コマンド

```
# クラスター一覧
kind get clusters

# クラスタの削除
kind delete cluster --name my-cluster

# kubeconfigのエクスポート
kind get kubeconfig --name my-cluster
```

2.5 k3d（k3s in Docker）

k3d は Rancher 社が開発した軽量 Kubernetes ディストリビューション k3s を Docker 上で実行するツールです。起動が非常に高速で、ローカルレジストリとの連携も簡単です。

2.5.1 インストール

```
# macOS / Linux
brew install k3d

# インストールスクリプト
curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
```

2.5.2 クラスタの作成

```
# シンプルなクラスタ作成
k3d cluster create my-cluster

# ポートマッピング付きで作成
k3d cluster create my-cluster -p "8080:80@loadbalancer"
```

```
# マルチノード
k3d cluster create my-cluster --servers 1 --agents 3
```

2.5.3 ローカルレジストリとの連携

k3d ではローカルレジストリを簡単に構築できます。

```
# レジストリ付きクラスタを作成
k3d cluster create my-cluster --registry-create my-registry:5000

# イメージをプッシュ
docker tag my-app:latest localhost:5000/my-app:latest
docker push localhost:5000/my-app:latest
```

2.5.4 管理コマンド

```
# クラスター一覧
k3d cluster list

# クラスタの停止・再開
k3d cluster stop my-cluster
k3d cluster start my-cluster

# クラスタの削除
k3d cluster delete my-cluster
```

2.6 k3s

k3s は Rancher 社が開発した軽量 Kubernetes ディストリビューションです。バイナリ 1 つで動作し、ARM 対応でエッジコンピューティングや IoT 環境にも適しています。本番環境でも利用されています。

2.6.1 インストール

```
# サーバー（コントロールプレーン）のインストール
curl -sfL https://get.k3s.io | sh -

# ワーカーノードの追加
curl -sfL https://get.k3s.io | K3S_URL=https://<サーバーIP>:6443 \
  K3S_TOKEN=<トークン> sh -
```

サーバーのトークンは以下で確認できます。

```
sudo cat /var/lib/rancher/k3s/server/node-token
```

2.6.2 k3s の特徴

- バイナリサイズが約 70MB と軽量
- SQLite、MySQL、PostgreSQL、etcd をデータストアとして選択可能
- Traefik（Ingress Controller）と ServiceLB（旧 Klipper）が組み込み済み
- containerd を内蔵（Docker は不要）
- CNCF の認定 Kubernetes ディストリビューション

2.6.3 kubectl の使用

k3s では kubectl が組み込まれています。

```
# k3s経由でkubectlを使用
sudo k3s kubectl get nodes

# 通常のkubectlを使用する場合はkubeconfigをコピー
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
kubectl get nodes
```

2.7 MicroK8s

MicroK8s は Canonical 社が開発した Snap パッケージで配布される Kubernetes ディストリビューションです。Ubuntu 環境で特に便利です。

2.7.1 インストール

```
# Ubuntu / Snap対応Linux
sudo snap install microk8s --classic

# ユーザーをmicrok8sグループに追加
sudo usermod -a -G microk8s $USER
newgrp microk8s
```

2.7.2 アドオンの有効化

MicroK8s はアドオン方式で機能を追加します。

```
# よく使うアドオンを一括有効化
microk8s enable dns dashboard storage ingress

# アドオンの一覧
microk8s status
```

2.7.3 管理コマンド

```
# 状態確認
microk8s status

# 起動・停止
microk8s start
microk8s stop

# kubectl（microk8s経由）
microk8s kubectl get nodes

# 通常のkubectlを使う場合
microk8s config > ~/.kube/config
```

2.8 Docker Desktop

Docker Desktop には Kubernetes が組み込まれており、設定画面からワンクリックで有効化できます。

2.8.1 有効化の手順

1. Docker Desktop を起動する
2. Settings（設定）を開く
3. 「Kubernetes」タブを選択する
4. 「Enable Kubernetes」にチェックを入れる
5. 「Apply & Restart」をクリックする

2.8.2 コンテキストの切り替え

Docker Desktop と他の Kubernetes 環境を併用する場合、コンテキストを切り替えます。

```
# 利用可能なコンテキストを一覧表示
kubectl config get-contexts

# Docker Desktop のコンテキストに切り替え
kubectl config use-context docker-desktop
```

2.9 ツールの選び方

用途	推奨ツール
Kubernetes の学習	Minikube（豊富なアドオンとドキュメント）
CI/CD でのテスト	kind または k3d（軽量で高速起動）
マルチノード検証	kind または k3d（設定ファイルで柔軟に構成）
エッジ / IoT	k3s（超軽量、ARM 対応）

用途	推奨ツール
Ubuntu 環境での開発	MicroK8s（Snap で簡単管理）
GUI 重視・初心者	Docker Desktop（ワンクリック有効化）
本番に近い環境のテスト	k3s または kubeadm

2.10 kubectl の基本操作

どのツールを選んでも、kubectl の操作は共通です。

2.10.1 ノードの確認

```
kubectl get nodes
```

2.10.2 全リソースの確認

```
kubectl get all
```

2.10.3 名前空間の確認

```
kubectl get namespaces
```

2.10.4 コンテキストの管理

複数のクラスタを使い分ける場合、kubectl のコンテキストを切り替えます。

```
# コンテキスト一覧
kubectl config get-contexts

# コンテキスト切り替え
kubectl config use-context <コンテキスト名>

# 現在のコンテキストを確認
kubectl config current-context
```

3 基本概念

3.1 Kubernetes のアーキテクチャ

Kubernetes クラスタは、大きく分けて**コントロールプレーン**と**ワーカーノード**の 2 つの要素で構成されます。

3.1.1 コントロールプレーン

クラスタ全体を管理する頭脳の役割を担います。

コンポーネント	役割
kube-apiserver	Kubernetes API を提供するフロントエンド。kubectl からのリクエストを受け付ける
etcd	クラスタの全データを保存する分散キーバリューストア
kube-scheduler	新しい Pod をどのノードに配置するか決定する
kube-controller-manager	各種コントローラー（ReplicaSet、Deployment など）を実行する

3.1.2 ワーカーノード

実際にコンテナが動作するマシンです。

コンポーネント	役割
kubelet	ノード上で Pod の起動・管理を行うエージェント
kube-proxy	ネットワークルールを管理し、Service への通信を転送する
コンテナランタイム	コンテナを実行する（containerd、CRI-O など）

3.2 主要なリソース

Kubernetes では、すべてのものが「リソース」として管理されます。以下が主要なリソースです。

3.2.1 ワークロード系リソース

- **Pod**: コンテナの最小デプロイ単位
- **ReplicaSet**: Pod のレプリカ数を維持する
- **Deployment**: ReplicaSet を管理し、ローリングアップデートを実現する
- **DaemonSet**: 各ノードに Pod を 1 つずつ配置する
- **StatefulSet**: ステートフルなアプリケーション向けの Pod 管理
- **Job / CronJob**: 一回限り、または定期的なバッチ処理を実行する

3.2.2 ネットワーク系リソース

- **Service**: Pod へのネットワークアクセスを提供する

- **Ingress**: HTTP/HTTPS のルーティングを管理する

3.2.3 設定・ストレージ系リソース

- **ConfigMap**: 設定データを管理する
- **Secret**: 機密情報を管理する
- **PersistentVolume (PV)**: 永続ストレージを提供する
- **PersistentVolumeClaim (PVC)**: PV の利用を要求する

3.2.4 クラスタ管理系リソース

- **Namespace**: クラスタ内のリソースを論理的に分離する
- **ServiceAccount**: Pod 用の認証情報を管理する
- **Role / ClusterRole**: アクセス権限を定義する

3.3 マニフェストファイル

Kubernetes ではリソースの定義を YAML ファイル（マニフェスト）で記述します。すべてのマニフェストには以下の共通フィールドがあります。

```yaml
apiVersion: v1          # APIのバージョン
kind: Pod               # リソースの種類
metadata:               # メタデータ
  name: my-pod          # リソース名
  labels:               # ラベル（任意のキー・バリュー）
    app: my-app
spec:                   # リソースの仕様（種類によって異なる）
  ...
```

3.3.1 apiVersion

リソースの種類によって使用する API バージョンが異なります。

リソース	apiVersion
Pod, Service, ConfigMap, Secret	`v1`
Deployment, ReplicaSet	`apps/v1`
Ingress	`networking.k8s.io/v1`
Job, CronJob	`batch/v1`

3.4 ラベルとセレクタ

3.4.1 ラベル

ラベルはリソースに付与するキー・バリューのペアです。リソースの分類やフィルタリングに使用します。

```yaml
metadata:
  labels:
    app: web-server
    env: production
    tier: frontend
```

3.4.2 セレクタ

セレクタはラベルを使ってリソースを選択する仕組みです。Deployment が Pod を管理したり、Service が Pod にトラフィックを送る際に使われます。

```yaml
selector:
  matchLabels:
    app: web-server
```

3.5 kubectl の基本コマンド

コマンド	説明
`kubectl get <リソース>`	リソースの一覧を表示する
`kubectl describe <リソース> <名前>`	リソースの詳細を表示する
`kubectl apply -f <ファイル>`	マニフェストを適用する（作成・更新）
`kubectl delete -f <ファイル>`	マニフェストのリソースを削除する
`kubectl logs <Pod名>`	Pod のログを表示する
`kubectl exec -it <Pod名> -- <コマンド>`	Pod 内でコマンドを実行する
`kubectl port-forward <Pod名> <ローカルポート>:<Podポート>`	Pod へポートフォワーディングする

3.5.1 出力フォーマット

`-o` オプションで出力形式を変更できます。

```bash
kubectl get pods -o wide          # 詳細情報を表示
kubectl get pods -o yaml          # YAML形式で出力
kubectl get pods -o json          # JSON形式で出力
kubectl get pods -o name          # リソース名のみ表示
```

4 Pod

4.1 Pod とは

Pod は Kubernetes におけるデプロイの最小単位です。1 つ以上のコンテナをまとめたグループであり、同一 Pod 内のコンテナはネットワークとストレージを共有します。

多くの場合、1 つの Pod には 1 つのコンテナを配置しますが、密接に連携する複数のコンテナを同一 Pod に配置するパターン（サイドカーパターン）もあります。

4.2 Pod の作成

4.2.1 マニフェストで作成

`nginx-pod.yaml` というファイルを作成します。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

マニフェストを適用して Pod を作成します。

```bash
kubectl apply -f nginx-pod.yaml
```

4.2.2 コマンドで直接作成

簡単なテスト用途では `kubectl run` コマンドでも作成できます。

```bash
kubectl run nginx-pod --image=nginx:1.27 --port=80
```

4.3 Pod の状態確認

4.3.1 一覧表示

```
kubectl get pods
```

```
NAME        READY   STATUS    RESTARTS   AGE
nginx-pod   1/1     Running   0          30s
```

4.3.2 詳細情報

```
kubectl describe pod nginx-pod
```

Pod の詳細情報が表示されます。トラブルシューティングの際に特に重要な情報は以下の通りです。

- **Events**: Pod に関するイベント（スケジュール、イメージのプル、起動など）
- **Conditions**: Pod の状態条件
- **Containers**: 各コンテナの状態

4.4 Pod のライフサイクル

Pod は以下の状態（Phase）を遷移します。

状態	説明
Pending	Pod が作成されたがまだコンテナが起動していない
Running	少なくとも 1 つのコンテナが実行中
Succeeded	すべてのコンテナが正常に終了した
Failed	少なくとも 1 つのコンテナが異常終了した
Unknown	Pod の状態を取得できない

4.5 Pod 内でのコマンド実行

```
# Podのシェルに接続
kubectl exec -it nginx-pod -- /bin/bash

# 単一コマンドの実行
kubectl exec nginx-pod -- cat /etc/nginx/nginx.conf
```

4.6 Pod のログ確認

```
# ログを表示
kubectl logs nginx-pod

# リアルタイムでログを追跡
kubectl logs -f nginx-pod

# 直近の100行を表示
kubectl logs --tail=100 nginx-pod
```

4.7 ポートフォワーディング

Pod に直接アクセスするにはポートフォワーディングを使用します。

```
kubectl port-forward nginx-pod 8080:80
```

ブラウザで `http://localhost:8080` にアクセスすると nginx の画面が表示されます。

4.8 マルチコンテナ Pod

1 つの Pod に複数のコンテナを配置する場合の例です。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: multi-container-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      ports:
        - containerPort: 80
    - name: sidecar
      image: busybox:1.36
      command: ["sh", "-c", "while true; do echo sidecar running; sleep 30; done"]
```

マルチコンテナ Pod ではコンテナ名を指定してログを確認します。

```
kubectl logs multi-container-pod -c sidecar
```

4.9 リソース制限

コンテナに対して CPU とメモリの制限を設定できます。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: resource-limited-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      resources:
        requests:          # 最低限確保するリソース
          memory: "64Mi"
          cpu: "250m"
        limits:            # 使用上限
          memory: "128Mi"
          cpu: "500m"
```

- **requests**: スケジューラがノードを選択する際の基準。この量のリソースが利用可能なノードに配置される
- **limits**: コンテナが使用できるリソースの上限。超過すると OOMKill（メモリ）やスロットリング（CPU）が発生する

4.10 ヘルスチェック

Kubernetes は 3 種類のヘルスチェック（Probe）をサポートしています。

4.10.1 livenessProbe

コンテナが正常に動作しているか確認します。失敗するとコンテナが再起動されます。

4.10.2 readinessProbe

コンテナがリクエストを受け付けられる状態か確認します。失敗すると Service のエンドポイントから除外されます。

4.10.3 startupProbe

コンテナの起動が完了したか確認します。起動が遅いアプリケーションに使用します。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: health-check-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      livenessProbe:
        httpGet:
```

```yaml
          path: /
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 3
        periodSeconds: 5
```

4.11 Pod の削除

```bash
# 名前を指定して削除
kubectl delete pod nginx-pod

# マニフェストで削除
kubectl delete -f nginx-pod.yaml
```

5 Deployment

5.1 Deployment とは

Deployment は、Pod のデプロイと管理を行うリソースです。Pod を直接管理するのではなく、ReplicaSet を通じて Pod を管理します。Deployment を使うことで、以下の機能が利用できます。

- Pod のレプリカ数の維持
- ローリングアップデート
- ロールバック
- スケーリング

実運用では Pod を直接作成せず、Deployment を使うのが一般的です。

5.2 Deployment と ReplicaSet の関係

Deployment の管理構造は以下のようになっています。

```
Deployment
  └── ReplicaSet
        ├── Pod
        ├── Pod
        └── Pod
```

- **Deployment**: 望ましい状態を宣言し、ReplicaSet を管理する
- **ReplicaSet**: 指定された数の Pod レプリカを維持する
- **Pod**: 実際にコンテナが動作する単位

5.3 Deployment の作成

`nginx-deployment.yaml` を作成します。

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
```

```
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

マニフェストの主要フィールドを解説します。

- `spec.replicas`: 維持する Pod の数
- `spec.selector`: 管理対象の Pod をラベルで選択
- `spec.template`: 作成する Pod のテンプレート（ラベルは selector と一致させる）

```
kubectl apply -f nginx-deployment.yaml
```

5.4 状態確認

```
# Deploymentの確認
kubectl get deployments

# ReplicaSetの確認
kubectl get replicasets

# Podの確認
kubectl get pods
```

```
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           30s
```

- **READY**: 準備完了の Pod 数 / 期待する Pod 数
- **UP-TO-DATE**: 最新の Pod テンプレートで作成された Pod 数
- **AVAILABLE**: 利用可能な Pod 数

5.5 スケーリング

5.5.1 コマンドでスケーリング

```
# レプリカ数を5に変更
kubectl scale deployment nginx-deployment --replicas=5

# 確認
kubectl get pods
```

5.5.2 マニフェストでスケーリング

`spec.replicas` の値を変更して `kubectl apply` を実行します。

```yaml
spec:
  replicas: 5
```

```
kubectl apply -f nginx-deployment.yaml
```

5.6 ローリングアップデート

Deployment の最大の利点の 1 つが、ダウンタイムなしのローリングアップデートです。

5.6.1 イメージの更新

```
# コマンドでイメージを更新
kubectl set image deployment/nginx-deployment nginx=nginx:1.27.3
```

またはマニフェストのイメージタグを変更して `kubectl apply` を実行します。

5.6.2 アップデートの進行状況を確認

```
kubectl rollout status deployment/nginx-deployment
```

```
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 3 new replicas
have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 2 out of 3 new replicas
have been updated...
deployment "nginx-deployment" successfully rolled out
```

5.6.3 アップデート戦略の設定

```yaml
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1           # 同時に追加できるPod数
      maxUnavailable: 0     # 同時に停止できるPod数
```

- `maxSurge`：アップデート中に追加で作成できる Pod 数（またはパーセント）
- `maxUnavailable`：アップデート中に利用不可にできる Pod 数（またはパーセント）

5.7 ロールバック

5.7.1 ロールアウト履歴の確認

```
kubectl rollout history deployment/nginx-deployment
```

```
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

5.7.2 特定リビジョンの詳細を確認

```
kubectl rollout history deployment/nginx-deployment --revision=1
```

5.7.3 前のバージョンに戻す

```
kubectl rollout undo deployment/nginx-deployment
```

5.7.4 特定のリビジョンに戻す

```
kubectl rollout undo deployment/nginx-deployment --to-revision=1
```

5.8 一時停止と再開

大きな変更を複数回に分けて行いたい場合、ロールアウトを一時停止できます。

```
# ロールアウトを一時停止
kubectl rollout pause deployment/nginx-deployment

# 複数の変更を適用
kubectl set image deployment/nginx-deployment nginx=nginx:1.27.3
kubectl set resources deployment/nginx-deployment -c=nginx --
limits=cpu=200m,memory=512Mi

# ロールアウトを再開（一度にデプロイされる）
kubectl rollout resume deployment/nginx-deployment
```

5.9 Deployment の削除

```
# 名前で削除
kubectl delete deployment nginx-deployment
```

```
# マニフェストで削除
kubectl delete -f nginx-deployment.yaml
```

Deployment を削除すると、管理下の ReplicaSet と Pod も自動的に削除されます。

6 Service

6.1 Service とは

Service は、Pod への安定したネットワークアクセスを提供するリソースです。Pod は起動するたびに IP アドレスが変わりますが、Service を使うことで固定のエンドポイントを通じて Pod にアクセスできます。

Service はラベルセレクタを使って対象の Pod を動的に選択し、トラフィックをロードバランシングします。

6.2 Service の種類

種類	説明	用途
ClusterIP	クラスタ内部のみでアクセス可能な仮想 IP を割り当てる	内部通信
NodePort	各ノードのポートを通じて外部からアクセス可能にする	開発・テスト
LoadBalancer	外部ロードバランサーを使って公開する	本番環境
ExternalName	外部の DNS 名にマッピングする	外部サービス連携

6.3 ClusterIP Service

クラスタ内部でのみアクセスできる Service です。デフォルトの Service 種類です。

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80          # Serviceのポート
      targetPort: 80    # Podのポート
```

```
kubectl apply -f nginx-service.yaml
```

6.3.1 確認

```
kubectl get services
```

```
NAME            TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
nginx-service   ClusterIP   10.96.42.123   <none>        80/TCP    30s
```

6.3.2 クラスタ内からのアクセス

クラスタ内の Pod から Service にアクセスするには、Service 名をホスト名として使用します。

```
# 一時的なPodからcurlでアクセス
kubectl run curl-test --image=curlimages/curl --rm -it -- curl http://nginx-service
```

6.4 NodePort Service

各ノードの指定されたポートを通じて外部からアクセスできる Service です。

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080    # ノードのポート（30000-32767）
```

NodePort は 30000〜32767 の範囲で指定できます。省略するとランダムに割り当てられます。

6.4.1 Minikube でのアクセス

```
minikube service nginx-nodeport --url
```

6.5 LoadBalancer Service

クラウド環境（AWS、GCP、Azure など）で外部ロードバランサーを自動的にプロビジョニングする Service です。

```
apiVersion: v1
kind: Service
```

```yaml
metadata:
  name: nginx-loadbalancer
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Minikube で使用する場合は、別のターミナルで `minikube tunnel` を実行する必要があります。

```bash
# 別のターミナルで実行
minikube tunnel
```

6.6 DNS によるサービスディスカバリ

Kubernetes クラスタ内では DNS が自動的に設定されます。以下の形式で Service にアクセスできます。

```
<Service名>.<Namespace>.svc.cluster.local
```

同じ Namespace 内であれば Service 名だけでアクセスできます。

```bash
# 同じNamespace内
curl http://nginx-service

# 別のNamespaceのServiceにアクセス
curl http://nginx-service.default.svc.cluster.local
```

6.7 複数ポートの公開

1 つの Service で複数のポートを公開できます。

```yaml
apiVersion: v1
kind: Service
metadata:
  name: multi-port-service
spec:
  selector:
    app: my-app
  ports:
    - name: http
      protocol: TCP
```

```
      port: 80
      targetPort: 8080
    - name: https
      protocol: TCP
      port: 443
      targetPort: 8443
```

複数ポートを定義する場合は、各ポートに `name` を指定する必要があります。

6.8 Headless Service

ClusterIP を割り当てず、DNS で Pod の IP アドレスを直接返す Service です。StatefulSet と組み合わせて使用されます。

```yaml
apiVersion: v1
kind: Service
metadata:
  name: headless-service
spec:
  clusterIP: None
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 80
```

6.9 Service の確認と削除

```bash
# Serviceの一覧
kubectl get services

# Serviceの詳細（エンドポイント情報を含む）
kubectl describe service nginx-service

# エンドポイントの確認
kubectl get endpoints nginx-service

# 削除
kubectl delete service nginx-service
```

7 ConfigMap と Secret

7.1 設定管理の必要性

アプリケーションの設定をコンテナイメージに直接埋め込むと、環境ごとにイメージを作り直す必要があります。Kubernetes では ConfigMap と Secret を使って設定をコンテナから分離し、柔軟に管理できます。

7.2 ConfigMap

ConfigMap は、機密性のない設定データをキー・バリューのペアで保存するリソースです。

7.2.1 ConfigMap の作成

7.2.1.1 マニフェストで作成

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  APP_DEBUG: "false"
  APP_LOG_LEVEL: "info"
```

```bash
kubectl apply -f app-config.yaml
```

7.2.1.2 コマンドで作成

```bash
# リテラル値から作成
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=APP_DEBUG=false

# ファイルから作成
kubectl create configmap nginx-config --from-file=nginx.conf

# ディレクトリから作成
kubectl create configmap configs --from-file=./config-dir/
```

7.2.2 ConfigMap の確認

```bash
kubectl get configmaps
kubectl describe configmap app-config
```

7.2.3 ConfigMap の利用方法

7.2.3.1 環境変数として利用

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: my-app:1.0
      envFrom:
        - configMapRef:
            name: app-config
```

`envFrom` を使うと、ConfigMap の全エントリが環境変数として設定されます。

特定のキーだけを使う場合は以下のように記述します。

```yaml
env:
  - name: APP_ENVIRONMENT
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_ENV
```

7.2.3.2 ボリュームとしてマウント

設定ファイルをそのままマウントしたい場合に使用します。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      volumeMounts:
        - name: config-volume
          mountPath: /etc/nginx/conf.d
  volumes:
    - name: config-volume
      configMap:
        name: nginx-config
```

7.3 Secret

Secret は、パスワード、API キー、証明書などの機密情報を保存するリソースです。Base64 でエンコードされて保存されます。

7.3.1 Secret の種類

種類	用途
Opaque	任意のデータ（デフォルト）
kubernetes.io/dockerconfigjson	Docker レジストリの認証情報
kubernetes.io/tls	TLS 証明書と秘密鍵
kubernetes.io/basic-auth	Basic 認証の資格情報

7.3.2 Secret の作成

7.3.2.1 マニフェストで作成

値は Base64 でエンコードする必要があります。

```
echo -n 'mypassword' | base64
# bXlwYXNzd29yZA==
```

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_PASSWORD: bXlwYXNzd29yZA==
  API_KEY: c2VjcmV0LWtleQ==
```

`stringData` フィールドを使うと Base64 エンコードなしで記述できます。

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:
  DB_PASSWORD: "mypassword"
  API_KEY: "secret-key"
```

7.3.2.2 コマンドで作成

```
kubectl create secret generic app-secret \
  --from-literal=DB_PASSWORD=mypassword \
  --from-literal=API_KEY=secret-key
```

7.3.3 Secret の利用方法

7.3.3.1 環境変数として利用

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: my-app:1.0
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: DB_PASSWORD
```

7.3.3.2 ボリュームとしてマウント

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: my-app:1.0
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/secrets
          readOnly: true
  volumes:
    - name: secret-volume
      secret:
        secretName: app-secret
```

7.4 Secret の注意点

- Secret の Base64 エンコードは暗号化ではありません。誰でもデコードできます
- Secret をマニフェストに含める場合、Git リポジトリにコミットしないよう注意してください

- 本番環境では External Secrets や Sealed Secrets などのツールの導入を検討してください
- etcd の暗号化を有効にすることで、保存時のセキュリティを強化できます

8 ストレージ

8.1 コンテナのストレージの課題

コンテナのファイルシステムは一時的なものです。コンテナが再起動されるとデータは失われます。データベースやファイルアップロードなど、永続的なデータ保存が必要な場合は Kubernetes のストレージ機能を使用します。

8.2 Volume

Volume は、Pod 内のコンテナ間でデータを共有したり、一時的なデータを保存するための仕組みです。Pod のライフサイクルに Volume 連動します。

8.2.1 emptyDir

Pod が作成されると空のディレクトリが作成され、Pod が削除されると消えます。同一 Pod 内のコンテナ間でデータを共有するために使用します。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: shared-data-pod
spec:
  containers:
    - name: writer
      image: busybox:1.36
      command: ["sh", "-c", "echo 'Hello' > /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
    - name: reader
      image: busybox:1.36
      command: ["sh", "-c", "cat /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
  volumes:
    - name: shared-data
      emptyDir: {}
```

8.2.2 hostPath

ノードのファイルシステム上のパスを Pod にマウントします。開発・テスト用途で使用しますが、本番環境での使用は推奨されません。

```yaml
volumes:
  - name: host-data
    hostPath:
      path: /data
      type: DirectoryOrCreate
```

8.3 PersistentVolume（PV）と PersistentVolumeClaim（PVC）

永続的なストレージを使用するには、PersistentVolume（PV）と PersistentVolumeClaim（PVC）を使用します。

- **PV**: クラスタ管理者がプロビジョニングするストレージリソース
- **PVC**: ユーザーがストレージを要求するためのリソース

8.3.1 PV の作成

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /mnt/data
```

8.3.2 アクセスモード

モード	略称	説明
ReadWriteOnce	RWO	単一ノードからの読み書き
ReadOnlyMany	ROX	複数ノードからの読み取り専用
ReadWriteMany	RWX	複数ノードからの読み書き

8.3.3 再利用ポリシー

ポリシー	説明
Retain	PVC が削除されても PV とデータを保持する
Delete	PVC が削除されると PV とデータも削除する
Recycle	データを削除して PV を再利用する（非推奨）

8.3.4 PVC の作成

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 3Gi
```

PVC を作成すると、条件に合う PV が自動的にバインドされます。

8.3.5 PVC を Pod で使用

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pvc-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      volumeMounts:
        - name: persistent-storage
          mountPath: /usr/share/nginx/html
  volumes:
    - name: persistent-storage
      persistentVolumeClaim:
        claimName: my-pvc
```

8.4 StorageClass

StorageClass を使うと、PVC が作成されたときに動的に PV をプロビジョニングできます（動的プロビジョニング）。

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: k8s.io/minikube-hostpath
parameters:
  type: pd-ssd
reclaimPolicy: Delete
```

PVC で StorageClass を指定します。

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dynamic-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast-storage
  resources:
    requests:
      storage: 10Gi
```

8.5 ストレージの確認

```bash
# PVの確認
kubectl get pv

# PVCの確認
kubectl get pvc

# StorageClassの確認
kubectl get storageclass
```

9 Namespace

9.1 Namespace とは

Namespace は、Kubernetes クラスタ内のリソースを論理的に分離するための仕組みです。同じクラスタ内で複数の環境（開発、ステージング、本番）やチームを分離する際に使用します。

9.2 デフォルトの Namespace

Kubernetes クラスタには以下の Namespace がデフォルトで存在します。

Namespace	説明
default	Namespace を指定しない場合に使用される
kube-system	Kubernetes システムコンポーネントが動作する
kube-public	全ユーザーが読み取り可能な公開リソース
kube-node-lease	ノードのハートビート情報を管理する

9.3 Namespace の操作

9.3.1 作成

```
# コマンドで作成
kubectl create namespace development

# マニフェストで作成
kubectl apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: development
  labels:
    env: dev
EOF
```

9.3.2 一覧表示

```
kubectl get namespaces
```

9.3.3 特定の Namespace 内のリソースを表示

```
# -n オプションでNamespaceを指定
kubectl get pods -n development
```

```
# すべてのNamespaceのリソースを表示
kubectl get pods --all-namespaces
# または
kubectl get pods -A
```

9.3.4 デフォルト Namespace の変更

kubectl で毎回 `-n` を指定するのが面倒な場合、デフォルトの Namespace を変更できます。

```
kubectl config set-context --current --namespace=development
```

9.4 Namespace にリソースを作成

マニフェストの `metadata.namespace` で Namespace を指定します。

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: development
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

```
kubectl apply -f nginx-deployment.yaml
```

9.5 ResourceQuota

Namespace ごとにリソースの使用量を制限できます。

```yaml
apiVersion: v1
kind: ResourceQuota
```

```yaml
metadata:
  name: dev-quota
  namespace: development
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: "8Gi"
    limits.cpu: "8"
    limits.memory: "16Gi"
```

9.5.1 ResourceQuota の確認

```bash
kubectl describe resourcequota dev-quota -n development
```

9.6 LimitRange

Namespace 内のコンテナに対してデフォルトのリソース制限を設定できます。

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: dev-limits
  namespace: development
spec:
  limits:
    - default:
        cpu: "500m"
        memory: "256Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
      max:
        cpu: "2"
        memory: "1Gi"
      min:
        cpu: "50m"
        memory: "64Mi"
      type: Container
```

ResourceQuota が設定されている場合、各 Pod には resources の requests と limits の指定が必須になります。LimitRange でデフォルト値を設定しておくと便利です。

9.7 Namespace の削除

```
kubectl delete namespace development
```

Namespace を削除すると、その Namespace 内のすべてのリソースも削除されます。削除前に必ず確認してください。

10 Ingress

10.1 Ingress とは

Ingress は、クラスタ外部からの HTTP/HTTPS トラフィックをクラスタ内の Service にルーティングするリソースです。Service の `NodePort` や `LoadBalancer` と異なり、URL パスベースやホスト名ベースのルーティング、TLS 終端などの高度な機能を提供します。

10.2 Ingress Controller

Ingress リソースを定義しただけではルーティングは機能しません。実際にトラフィックを処理する Ingress Controller が必要です。

10.2.1 主な Ingress Controller

名前	特徴	開発元
NGINX Ingress Controller	最も広く使われている、安定した実績	Kubernetes 公式
Traefik	自動設定、Let's Encrypt 対応、k3s に組み込み	Traefik Labs
HAProxy Ingress	高パフォーマンス、詳細な設定が可能	HAProxy
Istio Gateway	サービスメッシュと統合したルーティング	Istio
Contour	Envoy ベース、HTTPProxy カスタムリソース	VMware
AWS ALB Ingress Controller	AWS Application Load Balancer と統合	AWS

10.2.2 NGINX Ingress Controller のインストール

Minikube の場合はアドオンで簡単に有効化できます。

```
minikube addons enable ingress
```

Helm を使ってインストールする場合は以下の通りです。

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install ingress-nginx ingress-nginx/ingress-nginx
```

10.3 基本的な Ingress の定義

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```yaml
metadata:
  name: my-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: my-service
                port:
                  number: 80
```

```
kubectl apply -f my-ingress.yaml
```

10.4 パスベースのルーティング

1 つのホスト名で複数の Service にルーティングできます。

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: path-based-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 8080
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

10.4.1 pathType

pathType	説明
Prefix	URL パスのプレフィックスでマッチする（`/api` は `/api/users` にもマッチ）
Exact	URL パスが完全に一致する場合のみマッチする
ImplementationSpecific	Ingress Controller の実装に依存する

10.5 ホスト名ベースのルーティング

異なるホスト名で異なる Service にルーティングできます。

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: host-based-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: app-service
                port:
                  number: 80
    - host: api.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 8080
```

10.6 TLS 終端

Ingress で TLS（HTTPS）を終端できます。

10.6.1 TLS 用 Secret の作成

```
kubectl create secret tls my-tls-secret \
  --cert=tls.crt \
  --key=tls.key
```

10.6.2 Ingress の設定

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - myapp.example.com
      secretName: my-tls-secret
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: my-service
                port:
                  number: 80
```

10.7 cert-manager による自動 TLS 証明書管理

cert-manager を使うと、Let's Encrypt などの認証局から自動的に TLS 証明書を取得・更新できます。

10.7.1 cert-manager のインストール

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/latest/download/
cert-manager.yaml
```

10.7.2 ClusterIssuer の作成

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
```

```yaml
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: your-email@example.com
    privateKeySecretRef:
      name: letsencrypt-prod-key
    solvers:
      - http01:
          ingress:
            class: nginx
```

10.7.3 Ingress でのアノテーション

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: auto-tls-ingress
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - myapp.example.com
      secretName: myapp-tls
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: my-service
                port:
                  number: 80
```

10.8 NGINX Ingress のアノテーション

NGINX Ingress Controller では、アノテーションを使って細かい設定ができます。

```yaml
metadata:
  annotations:
    # リダイレクト
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    # リクエストサイズ制限
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
```

```
    # タイムアウト設定
    nginx.ingress.kubernetes.io/proxy-read-timeout: "60"
    # CORS設定
    nginx.ingress.kubernetes.io/enable-cors: "true"
    # レート制限
    nginx.ingress.kubernetes.io/limit-rps: "10"
```

10.9 Ingress の確認

```
# Ingressの一覧
kubectl get ingress

# 詳細情報
kubectl describe ingress my-ingress

# Ingress Controllerのログ
kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx
```

11 Helm

11.1 Helm とは

Helm は Kubernetes のパッケージマネージャです。複数の Kubernetes マニフェストを「Chart」というパッケージにまとめて管理します。Linux における apt や yum の Kubernetes 版と考えるとわかりやすいでしょう。

Helm を使うことで以下のメリットがあります。

- 複雑なアプリケーションをワンコマンドでデプロイできる
- 環境ごとの設定値（values）を簡単に切り替えられる
- アプリケーションのバージョン管理とロールバックができる
- 豊富なコミュニティ Chart を活用できる

11.2 インストール

```
# macOS
brew install helm

# Linux
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Windows
winget install Helm.Helm
```

```
# バージョン確認
helm version
```

11.3 Chart リポジトリ

11.3.1 リポジトリの追加

```
# 公式の安定版リポジトリを追加
helm repo add bitnami https://charts.bitnami.com/bitnami

# Prometheus用リポジトリを追加
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

# リポジトリの更新
helm repo update
```

11.3.2 リポジトリの管理

```
# リポジトリの一覧
helm repo list

# リポジトリの削除
helm repo remove bitnami
```

11.3.3 Chart の検索

```
# リポジトリ内のChartを検索
helm search repo nginx

# Artifact Hubで検索
helm search hub prometheus
```

11.4 Chart のインストール

11.4.1 基本的なインストール

```
# nginx をインストール
helm install my-nginx bitnami/nginx

# Namespace を指定してインストール
helm install my-nginx bitnami/nginx -n web --create-namespace
```

`helm install <リリース名> <Chart名>` の形式です。リリース名はクラスタ内で一意である必要があります。

11.4.2 インストール前の確認

```
# テンプレートを展開して確認（ドライラン）
helm install my-nginx bitnami/nginx --dry-run

# 生成されるマニフェストを確認
helm template my-nginx bitnami/nginx
```

11.4.3 設定値のカスタマイズ

Chart の設定値をカスタマイズするには `--set` フラグまたは `values.yaml` ファイルを使用します。

```
# --setで直接指定
helm install my-nginx bitnami/nginx \
  --set replicaCount=3 \
  --set service.type=NodePort
```

```
# values.yamlファイルで指定
helm install my-nginx bitnami/nginx -f my-values.yaml
```

`my-values.yaml` の例:

```yaml
replicaCount: 3
service:
  type: NodePort
  port: 80
resources:
  requests:
    memory: "128Mi"
    cpu: "100m"
  limits:
    memory: "256Mi"
    cpu: "200m"
```

11.4.4 利用可能な設定値を確認

```
# Chartのデフォルトvaluesを確認
helm show values bitnami/nginx

# Chartの情報を確認
helm show chart bitnami/nginx
```

11.5 リリースの管理

11.5.1 リリースの一覧

```
# 全リリースを表示
helm list

# 特定のNamespace
helm list -n web

# 全Namespace
helm list -A
```

11.5.2 アップグレード

```
# 設定値を変更してアップグレード
helm upgrade my-nginx bitnami/nginx --set replicaCount=5
```

```
# values.yamlを指定してアップグレード
helm upgrade my-nginx bitnami/nginx -f my-values.yaml

# インストールされていなければインストールする
helm upgrade --install my-nginx bitnami/nginx
```

11.5.3 ロールバック

```
# リリース履歴を確認
helm history my-nginx

# 前のバージョンに戻す
helm rollback my-nginx

# 特定のリビジョンに戻す
helm rollback my-nginx 2
```

11.5.4 アンインストール

```
helm uninstall my-nginx
```

11.6 独自 Chart の作成

11.6.1 Chart のひな形を作成

```
helm create my-chart
```

以下のディレクトリ構造が生成されます。

```
my-chart/
├── Chart.yaml          # Chartのメタデータ
├── values.yaml         # デフォルトの設定値
├── charts/             # 依存Chart
├── templates/          # マニフェストテンプレート
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── hpa.yaml
│   ├── serviceaccount.yaml
│   ├── _helpers.tpl    # テンプレートヘルパー
│   ├── NOTES.txt       # インストール後のメッセージ
│   └── tests/
│       └── test-connection.yaml
└── .helmignore
```

11.6.2 Chart.yaml

```yaml
apiVersion: v2
name: my-chart
description: A Helm chart for my application
type: application
version: 0.1.0          # Chartのバージョン
appVersion: "1.0.0"     # アプリケーションのバージョン
```

11.6.3 テンプレートの記法

Helm テンプレートでは Go テンプレート構文を使用します。

```yaml
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "my-chart.fullname" . }}
  labels:
    {{- include "my-chart.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "my-chart.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "my-chart.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: {{ .Values.service.port }}
```

11.6.4 ローカル Chart のインストール

```bash
# ディレクトリから直接インストール
helm install my-release ./my-chart

# パッケージ化してインストール
helm package my-chart
helm install my-release my-chart-0.1.0.tgz
```

11.7 よく使われるコミュニティ Chart

Chart	リポジトリ	用途
ingress-nginx	ingress-nginx	NGINX Ingress Controller
cert-manager	jetstack	TLS 証明書の自動管理
prometheus	prometheus-community	メトリクス収集・監視
grafana	grafana	ダッシュボード・可視化
argo-cd	argo	GitOps による CD
postgresql	bitnami	PostgreSQL データベース
redis	bitnami	Redis キャッシュ
sealed-secrets	sealed-secrets	暗号化された Secret 管理

12 Kustomize

12.1 Kustomize とは

Kustomize は、Kubernetes マニフェストをテンプレートなしでカスタマイズするツールです。ベースとなるマニフェストに対して「パッチ」や「オーバーレイ」を適用することで、環境ごとの差分を管理します。

Kustomize は kubectl に組み込まれており、追加のインストールなしで使用できます。

12.2 Helm と Kustomize の比較

特徴	Helm	Kustomize
アプローチ	テンプレートエンジン	オーバーレイ（パッチ）
学習コスト	Go テンプレート構文が必要	YAML のみで完結
パッケージ管理	Chart リポジトリで配布	Git で管理
ロールバック	helm rollback で可能	Git で管理
コミュニティ Chart	豊富に存在	なし（自分で作成）
kubectl 統合	別途インストール必要	kubectl に組み込み済み

Helm と Kustomize は競合するものではなく、併用されることも多いです。Helm でインストールしたものを Kustomize でカスタマイズするパターンもあります。

12.3 基本的なディレクトリ構造

```
my-app/
├── base/                    # ベースのマニフェスト
│   ├── kustomization.yaml
│   ├── deployment.yaml
│   └── service.yaml
└── overlays/                # 環境ごとのオーバーレイ
    ├── development/
    │   └── kustomization.yaml
    ├── staging/
    │   └── kustomization.yaml
    └── production/
        └── kustomization.yaml
```

12.4 ベースの作成

12.4.1 マニフェストファイル

```yaml
# base/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: my-app:latest
          ports:
            - containerPort: 8080
```

```yaml
# base/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app
spec:
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
```

12.4.2 kustomization.yaml

```yaml
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
commonLabels:
  managed-by: kustomize
```

12.4.3 ベースの適用

```
kubectl apply -k ./base/
```

12.5 オーバーレイの作成

12.5.1 開発環境

```yaml
# overlays/development/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
namePrefix: dev-
namespace: development
patches:
  - target:
      kind: Deployment
      name: my-app
    patch: |
      - op: replace
        path: /spec/replicas
        value: 1
```

12.5.2 本番環境

```yaml
# overlays/production/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
namePrefix: prod-
namespace: production
patches:
  - target:
      kind: Deployment
      name: my-app
    patch: |
      - op: replace
        path: /spec/replicas
        value: 5
      - op: replace
        path: /spec/template/spec/containers/0/image
        value: my-app:v1.2.3
```

12.5.3 オーバーレイの適用

```
# 開発環境に適用
kubectl apply -k ./overlays/development/

# 本番環境に適用
kubectl apply -k ./overlays/production/

# 適用前にマニフェストを確認
kubectl kustomize ./overlays/production/
```

12.6 主な機能

12.6.1 namePrefix / nameSuffix

リソース名にプレフィックスやサフィックスを追加します。

```
namePrefix: staging-
nameSuffix: -v2
```

12.6.2 namespace

すべてのリソースに Namespace を設定します。

```
namespace: staging
```

12.6.3 commonLabels / commonAnnotations

すべてのリソースに共通のラベルやアノテーションを追加します。

```
commonLabels:
  env: staging
  team: backend
commonAnnotations:
  note: "managed by kustomize"
```

12.6.4 images

コンテナイメージを変更します。マニフェストを直接編集せずにイメージタグを切り替えられます。

```
images:
  - name: my-app
    newName: registry.example.com/my-app
    newTag: v1.2.3
```

12.6.5 ConfigMapGenerator / SecretGenerator

ConfigMap や Secret を自動生成します。内容が変わるとハッシュ付きの名前が生成され、Pod の再起動がトリガーされます。

```yaml
configMapGenerator:
  - name: app-config
    literals:
      - APP_ENV=production
      - APP_LOG_LEVEL=warn
    files:
      - config.json

secretGenerator:
  - name: app-secret
    literals:
      - DB_PASSWORD=mypassword
    type: Opaque
```

12.6.6 patchesStrategicMerge

Strategic Merge パッチでマニフェストを部分的に上書きします。

```yaml
# overlays/production/increase-resources.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    spec:
      containers:
        - name: my-app
          resources:
            requests:
              cpu: "500m"
              memory: "256Mi"
            limits:
              cpu: "1000m"
              memory: "512Mi"
```

```yaml
# overlays/production/kustomization.yaml
patches:
  - path: increase-resources.yaml
```

12.7 kustomize コマンド

```
# ビルド結果を表示（適用せずに確認）
kubectl kustomize ./overlays/production/

# 直接適用
kubectl apply -k ./overlays/production/

# 削除
kubectl delete -k ./overlays/production/

# diff（現在の状態との差分を確認）
kubectl diff -k ./overlays/production/
```

13 ArgoCD と GitOps

13.1 GitOps とは

GitOps は、Git リポジトリをインフラとアプリケーションの信頼できる唯一の情報源（Single Source of Truth）として扱う運用手法です。Kubernetes クラスタの望ましい状態を Git で管理し、Git への変更が自動的にクラスタに反映されます。

13.1.1 GitOps の原則

- **宣言的設定**: システムの望ましい状態を宣言的に記述する
- **Git で管理**: すべての設定変更は Git を通じて行う
- **自動同期**: Git の状態とクラスタの状態を自動的に一致させる
- **自動修復**: クラスタの状態が Git と異なる場合、自動的に修正する

13.1.2 従来の CI/CD と GitOps の違い

観点	従来の CI/CD（Push 型）	GitOps（Pull 型）
デプロイ方式	CI パイプラインがクラスタに push	クラスタ側が Git から pull
認証情報	CI 側にクラスタの認証情報が必要	クラスタ内で完結
状態管理	デプロイ時点のみ把握	継続的に Git と同期
ドリフト検知	手動で確認	自動で検知・修正
監査	CI のログに依存	Git の履歴で完全に追跡可能

13.2 主な GitOps ツール

ツール	特徴	開発元
Argo CD	Web UI 付き、豊富な機能、CNCF の卒業プロジェクト	Intuit / CNCF
Flux	軽量、GitOps Toolkit ベース、CNCF の卒業プロジェクト	Weaveworks / CNCF

本章ではデファクトスタンダードとなっている Argo CD を中心に解説します。

13.3 Argo CD とは

Argo CD は、Kubernetes 用の宣言的な GitOps 型継続的デリバリー（CD）ツールです。Git リポジトリに格納されたマニフェストを Kubernetes クラスタに自動的にデプロイします。

13.3.1 Argo CD の主な特徴

- **Web UI**: アプリケーションの状態をビジュアルに確認できる

- **マルチクラスタ対応**: 複数のクラスタを一元管理できる
- **SSO 連携**: OIDC、OAuth2、LDAP、SAML に対応
- **RBAC**: きめ細かいアクセス制御
- **Webhook 対応**: GitHub などからの Webhook で即座に同期できる
- **Helm / Kustomize 対応**: 各種マニフェスト管理ツールと統合

13.4 Argo CD のインストール

13.4.1 kubectl でインストール

```
# Namespaceの作成
kubectl create namespace argocd

# Argo CDのインストール
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/
manifests/install.yaml

# Podが起動するまで待機
kubectl wait --for=condition=Ready pods --all -n argocd --timeout=300s
```

13.4.2 Helm でインストール

```
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update
helm install argocd argo/argo-cd -n argocd --create-namespace
```

13.5 Argo CD CLI のインストール

```
# macOS
brew install argocd

# Linux
curl -sSL -o argocd https://github.com/argoproj/argo-cd/releases/latest/download/
argocd-linux-amd64
chmod +x argocd
sudo mv argocd /usr/local/bin/
```

13.6 Web UI へのアクセス

```
# ポートフォワーディング
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

ブラウザで `https://localhost:8080` にアクセスします。

13.6.1 初期パスワードの取得

```
# 初期パスワードを取得
argocd admin initial-password -n argocd
```

ユーザー名は `admin` です。ログイン後はパスワードを変更してください。

```
# CLIでログイン
argocd login localhost:8080

# パスワード変更
argocd account update-password
```

13.7 Application の作成

Argo CD では「Application」リソースで Git リポジトリとクラスタの対応を定義します。

13.7.1 CLI で作成

```
argocd app create my-app \
  --repo https://github.com/your-org/your-repo.git \
  --path manifests \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace default \
  --sync-policy automated
```

13.7.2 マニフェストで作成

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/your-repo.git
    targetRevision: main
    path: manifests
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true         # Gitから削除されたリソースをクラスタからも削除
      selfHeal: true      # クラスタの手動変更を自動的に元に戻す
```

```
    syncOptions:
      - CreateNamespace=true
```

13.7.3 Helm Chart をソースにする場合

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: nginx
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://charts.bitnami.com/bitnami
    chart: nginx
    targetRevision: 15.0.0
    helm:
      values: |
        replicaCount: 3
        service:
          type: ClusterIP
  destination:
    server: https://kubernetes.default.svc
    namespace: web
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

13.7.4 Kustomize をソースにする場合

```
spec:
  source:
    repoURL: https://github.com/your-org/your-repo.git
    targetRevision: main
    path: overlays/production
    # Kustomize は自動的に検出される
```

13.8 同期（Sync）

13.8.1 同期の状態

状態	説明
Synced	Git とクラスタの状態が一致している
OutOfSync	Git とクラスタの状態が異なっている

状態	説明
Unknown	状態を判定できない

13.8.2 手動同期

```
# CLIで同期
argocd app sync my-app

# ドライラン（実際には適用しない）
argocd app sync my-app --dry-run
```

13.8.3 自動同期の設定

```
syncPolicy:
  automated:
    prune: true        # 不要なリソースを自動削除
    selfHeal: true     # 手動変更を自動修正
    allowEmpty: false  # すべてのリソースが削除される同期を防止
```

13.9 Application の管理

```
# アプリケーション一覧
argocd app list

# アプリケーションの詳細
argocd app get my-app

# 同期差分の確認
argocd app diff my-app

# アプリケーションの削除
argocd app delete my-app

# ロールバック
argocd app rollback my-app <リビジョン番号>

# アプリケーションの履歴
argocd app history my-app
```

13.10 ApplicationSet による複数環境管理

ApplicationSet を使うと、1 つのテンプレートから複数の Application を自動生成できます。

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: my-app-set
  namespace: argocd
spec:
  generators:
    - list:
        elements:
          - env: development
            namespace: dev
            revision: develop
          - env: staging
            namespace: staging
            revision: staging
          - env: production
            namespace: production
            revision: main
  template:
    metadata:
      name: "my-app-{{env}}"
    spec:
      project: default
      source:
        repoURL: https://github.com/your-org/your-repo.git
        targetRevision: "{{revision}}"
        path: overlays/{{env}}
      destination:
        server: https://kubernetes.default.svc
        namespace: "{{namespace}}"
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
```

13.11 GitOps のベストプラクティス

- **アプリケーションコードとマニフェストのリポジトリを分離する**: CI の変更が CD に意図せず影響するのを防ぐ
- **ブランチ戦略を明確にする**: 環境ごとのブランチまたはディレクトリ構造を統一する
- **Secret は暗号化して管理する**: Sealed Secrets や SOPS、External Secrets を使用する
- **PR ベースのデプロイフローを導入する**: マニフェストの変更は PR レビューを通じて行う
- **通知を設定する**: Slack などに同期結果を通知する

13.12 Flux（補足）

Flux も CNCF 卒業プロジェクトの GitOps ツールです。Argo CD との主な違いは以下の通りです。

- Web UI が組み込まれていない（別途 Weave GitOps UI などを使用）
- GitOps Toolkit によるモジュラーアーキテクチャ
- Kustomize Controller が組み込みで、Kustomize との親和性が高い
- Helm Controller により、Helm リリースも GitOps 管理できる

```
# Fluxのインストール
flux install

# GitリポジトリとFluxの接続
flux create source git my-repo \
  --url=https://github.com/your-org/your-repo \
  --branch=main

# Kustomizationの作成
flux create kustomization my-app \
  --source=my-repo \
  --path="./overlays/production" \
  --prune=true
```

14 モニタリング

14.1 Kubernetes におけるモニタリングの重要性

Kubernetes クラスタを運用するには、アプリケーションとインフラの状態を継続的に監視する必要があります。モニタリングにより、障害の早期検知、パフォーマンスのボトルネック特定、キャパシティプランニングが可能になります。

14.2 モニタリングスタック

Kubernetes のモニタリングにはデファクトスタンダードとなっているツールの組み合わせがあります。

ツール	役割	説明
Prometheus	メトリクス収集	時系列データベース、プル型のメトリクス収集
Grafana	可視化	ダッシュボードによるメトリクスの可視化
Alertmanager	アラート管理	Prometheus からのアラートをルーティング・通知
Loki	ログ収集	Prometheus ライクなログ集約システム
Promtail / Alloy	ログ転送	各ノードから Loki へログを転送

14.3 Prometheus

14.3.1 Prometheus とは

Prometheus は CNCF の卒業プロジェクトで、Kubernetes クラスタのメトリクス収集と保存を行う時系列データベースです。プル型でターゲットからメトリクスをスクレイプ（収集）します。

14.3.2 kube-prometheus-stack のインストール

Prometheus、Grafana、Alertmanager、各種 Exporter をまとめてインストールできる kube-prometheus-stack が広く使われています。

```
# Helmリポジトリの追加
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

# インストール
```

```
helm install prometheus prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace
```

14.3.3 インストールされるコンポーネント

```
kubectl get pods -n monitoring
```

- **prometheus-server**: メトリクスの収集と保存
- **alertmanager**: アラートのルーティングと通知
- **grafana**: ダッシュボード UI
- **node-exporter**: ノードのメトリクスを公開
- **kube-state-metrics**: Kubernetes リソースの状態をメトリクスとして公開

14.3.4 Prometheus UI へのアクセス

```
kubectl port-forward svc/prometheus-kube-prometheus-prometheus -n monitoring 9090:9090
```

ブラウザで `http://localhost:9090` にアクセスします。

14.3.5 PromQL（Prometheus Query Language）

Prometheus では PromQL というクエリ言語でメトリクスを検索・集計します。

```
# Podごとの CPU 使用率
rate(container_cpu_usage_seconds_total{namespace="default"}[5m])

# Podごとのメモリ使用量
container_memory_usage_bytes{namespace="default"}

# HTTPリクエストのレート
rate(http_requests_total[5m])

# 5xx エラー率
rate(http_requests_total{status=~"5.."}[5m])
/ rate(http_requests_total[5m])
```

14.3.6 ServiceMonitor

Prometheus Operator では、ServiceMonitor リソースを使ってスクレイプ対象を定義します。

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app-monitor
  namespace: monitoring
  labels:
```

```
    release: prometheus
spec:
  selector:
    matchLabels:
      app: my-app
  endpoints:
    - port: metrics
      interval: 30s
      path: /metrics
  namespaceSelector:
    matchNames:
      - default
```

アプリケーション側で `/metrics` エンドポイントを公開する必要があります。

14.4 Grafana

14.4.1 Grafana UI へのアクセス

```
kubectl port-forward svc/prometheus-grafana -n monitoring 3000:80
```

ブラウザで `http://localhost:3000` にアクセスします。

14.4.2 初期認証情報

kube-prometheus-stack でインストールした場合のデフォルト:

- ユーザー名: `admin`
- パスワード: `prom-operator`

14.4.3 プリインストールされたダッシュボード

kube-prometheus-stack には多数のダッシュボードがプリインストールされています。

- **Kubernetes / Compute Resources / Cluster**: クラスタ全体のリソース使用量
- **Kubernetes / Compute Resources / Namespace (Pods)**: Namespace 別の Pod リソース
- **Kubernetes / Compute Resources / Node (Pods)**: ノード別の Pod リソース
- **Node Exporter / Nodes**: ノードの CPU、メモリ、ディスク、ネットワーク

14.4.4 カスタムダッシュボードの作成

Grafana では独自のダッシュボードを作成できます。

1. 左メニューの「+」→「New Dashboard」を選択
2. 「Add visualization」をクリック
3. データソースに Prometheus を選択
4. PromQL クエリを入力してパネルを作成
5. ダッシュボードを保存

14.5 Alertmanager

14.5.1 アラートルールの定義

Prometheus Operator では、PrometheusRule リソースでアラートルールを定義します。

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: my-app-alerts
  namespace: monitoring
  labels:
    release: prometheus
spec:
  groups:
    - name: my-app
      rules:
        - alert: HighMemoryUsage
          expr: |
            container_memory_usage_bytes{namespace="default",container="my-app"}
            / container_spec_memory_limit_bytes{namespace="default",container="my-app"}
            > 0.8
          for: 5m
          labels:
            severity: warning
          annotations:
            summary: "High memory usage detected"
            description: "Pod {{ $labels.pod }} memory usage is above 80%"

        - alert: PodCrashLooping
          expr: rate(kube_pod_container_status_restarts_total{namespace="default"}
[15m]) > 0
          for: 5m
          labels:
            severity: critical
          annotations:
            summary: "Pod is crash looping"
            description: "Pod {{ $labels.pod }} has been restarting"
```

14.5.2 通知先の設定

Alertmanager で Slack やメールに通知を送信できます。Helm の values で設定します。

```yaml
# values.yaml（kube-prometheus-stack）
alertmanager:
  config:
    route:
      receiver: slack
      group_by: ['alertname', 'namespace']
```

```yaml
      group_wait: 30s
      group_interval: 5m
      repeat_interval: 4h
    receivers:
      - name: slack
        slack_configs:
          - channel: '#alerts'
            send_resolved: true
            api_url: 'https://hooks.slack.com/services/xxx/yyy/zzz'
```

14.6 Loki（ログ集約）

Loki は Grafana Labs が開発した、Prometheus に着想を得たログ集約システムです。ラベルベースでログをインデックス化するため、軽量で効率的です。

14.6.1 インストール

```bash
# Loki Stack（Loki + Promtail）のインストール
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
helm install loki grafana/loki-stack \
  -n monitoring \
  --set grafana.enabled=false \
  --set promtail.enabled=true
```

14.6.2 Grafana で Loki のログを確認

1. Grafana の左メニューから「Connections」→「Data sources」を選択
2. 「Add data source」で Loki を追加
3. URL に `http://loki:3100` を設定
4. 「Explore」でログを検索

14.6.3 LogQL（ログクエリ）

```
# Namespace でフィルタリング
{namespace="default"}

# Pod名でフィルタリング
{namespace="default", pod=~"my-app.*"}

# キーワードで検索
{namespace="default"} |= "error"

# 正規表現で検索
{namespace="default"} |~ "status=[45].."
```

```
# エラーログのレート
rate({namespace="default"} |= "error" [5m])
```

14.7 metrics-server

metrics-server は `kubectl top` コマンドや HPA（Horizontal Pod Autoscaler）に必要な軽量メトリクス収集ツールです。

```
# Minikubeの場合
minikube addons enable metrics-server

# Helmの場合
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm install metrics-server metrics-server/metrics-server -n kube-system
```

```
# ノードのリソース使用量
kubectl top nodes

# Podのリソース使用量
kubectl top pods

# 特定のNamespace
kubectl top pods -n monitoring
```

15 実践運用

15.1 アプリケーションのデプロイ例

実践的な例として、Web アプリケーションをデプロイする一連の流れを紹介します。

15.1.1 Deployment の作成

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web-app
          image: nginx:1.27
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "200m"
              memory: "256Mi"
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 5
```

15.1.2 Service の作成

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-app-service
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
```

15.1.3 ConfigMap の作成

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-app-config
data:
  APP_ENV: "production"
  APP_LOG_LEVEL: "warn"
```

15.1.4 一括適用

すべてのマニフェストを 1 つのディレクトリに配置して一括適用できます。

```bash
# ディレクトリ内のすべてのマニフェストを適用
kubectl apply -f ./manifests/

# 再帰的に適用
kubectl apply -f ./manifests/ -R
```

15.2 トラブルシューティング

15.2.1 Pod が起動しない場合

```bash
# Podの状態を確認
kubectl get pods

# 詳細情報とイベントを確認
kubectl describe pod <Pod名>
```

```
# ログを確認
kubectl logs <Pod名>

# 前回のコンテナのログを確認（再起動した場合）
kubectl logs <Pod名> --previous
```

15.2.2 よくある Pod のエラー

状態	原因	対処法
ImagePullBackOff	イメージの取得に失敗	イメージ名・タグの確認、レジストリの認証確認
CrashLoopBackOff	コンテナが繰り返しクラッシュ	ログの確認、アプリケーションの修正
Pending	スケジュールできない	ノードのリソース確認、taint の確認
OOMKilled	メモリ不足で強制終了	メモリの limits を増やす
CreateContainerConfigError	設定エラー	ConfigMap や Secret の存在確認

15.2.3 Service に接続できない場合

```
# Serviceの確認
kubectl get service <Service名>

# エンドポイントの確認（Podが紐づいているか）
kubectl get endpoints <Service名>

# ラベルの一致を確認
kubectl get pods --show-labels
```

15.3 リソースの監視

15.3.1 リソース使用量の確認

```
# ノードのリソース使用量
kubectl top nodes

# Podのリソース使用量
kubectl top pods
```

`kubectl top` を使用するには、metrics-server が必要です。Minikube では以下のコマンドで有効にできます。

```
minikube addons enable metrics-server
```

15.4 ラベルを活用した運用

15.4.1 ラベルの追加・変更

```
# ラベルを追加
kubectl label pods nginx-pod version=v1

# ラベルを上書き
kubectl label pods nginx-pod version=v2 --overwrite

# ラベルを削除
kubectl label pods nginx-pod version-
```

15.4.2 ラベルによるフィルタリング

```
# 特定のラベルを持つPodを表示
kubectl get pods -l app=nginx

# 複数条件でフィルタリング
kubectl get pods -l "app=nginx,env=production"

# ラベルの表示
kubectl get pods --show-labels
```

15.5 便利な kubectl コマンド

コマンド	説明
`kubectl get events --sort-by=.metadata.creationTimestamp`	イベントを時系列で表示
`kubectl api-resources`	利用可能なリソース種類を一覧表示
`kubectl explain <リソース>`	リソースのフィールドを確認
`kubectl diff -f <ファイル>`	適用前に差分を確認
`kubectl debug <Pod名> -it --image=busybox`	デバッグ用コンテナを起動

15.6 次のステップ

本ガイドでは Kubernetes の基礎からエコシステムの主要ツールまでを学びました。さらに学習を進めるために、以下のトピックを探索してみてください。

- **RBAC**: ロールベースアクセス制御
- **NetworkPolicy**: Pod 間のネットワーク制御
- **HPA（Horizontal Pod Autoscaler）**: 自動水平スケーリング
- **StatefulSet**: ステートフルアプリケーションの管理
- **Operator**: カスタムコントローラーの開発

- **サービスメッシュ**: Istio、Linkerd によるサービス間通信の管理
- **ポリシーエンジン**: OPA/Gatekeeper、Kyverno によるポリシー管理
- **Terraform**: クラスタ自体の Infrastructure as Code

15.6.1 参考リソース

- Kubernetes 公式ドキュメント: https://kubernetes.io/ja/docs/
- Kubernetes 公式チュートリアル: https://kubernetes.io/ja/docs/tutorials/
- kubectl チートシート: https://kubernetes.io/ja/docs/reference/kubectl/cheatsheet/
- Helm 公式ドキュメント: https://helm.sh/ja/docs/
- Argo CD 公式ドキュメント: https://argo-cd.readthedocs.io/
- Prometheus 公式ドキュメント: https://prometheus.io/docs/
- Grafana 公式ドキュメント: https://grafana.com/docs/

16 Kubernetes ディストリビューションとクラスタ構築

16.1 ディストリビューションの全体像

Kubernetes はオープンソースプロジェクトとして公開されていますが、本番環境でクラスタを構築・運用するには様々な選択肢があります。素の Kubernetes を自前で構築する方法から、追加機能が統合されたディストリビューション、クラウドのマネージドサービスまで、用途に応じて選択します。

カテゴリ	代表的なツール/サービス	管理負荷	推奨用途
公式構築ツール	kubeadm, Kubespray	高	学習・オンプレミス
宣言的プロビジョニング	Cluster API	中	マルチクラスタ管理
軽量ディストリビューション	k3s, k0s, MicroK8s	低〜中	エッジ・小規模本番
イミュータブル OS 型	Talos Linux	低	セキュリティ重視の本番
エンタープライズ	OpenShift, Rancher / RKE2, Tanzu	中	大規模組織・本番
コンテナ最適化 OS	Bottlerocket, Flatcar	低	マネージドノード
マネージドサービス	EKS, GKE, AKS	低	クラウド本番

16.2 kubeadm

16.2.1 kubeadm とは

kubeadm は Kubernetes 公式のクラスタ構築ツールです。Kubernetes のコアコンポーネントをベストプラクティスに沿ってブートストラップします。素の Kubernetes を構築するための標準的な方法であり、Kubernetes の内部構造を学ぶのにも最適です。

16.2.2 前提条件

- 2 台以上の Linux マシン（物理 or VM）
- 各ノード: 2CPU 以上、2GB 以上のメモリ
- コンテナランタイム（containerd 推奨）
- ネットワーク接続（ノード間で通信可能）
- swap 無効

16.2.3 コンテナランタイムのインストール

```
# containerdのインストール（Ubuntu）
sudo apt-get update
sudo apt-get install -y containerd

# 設定ファイルの生成
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml

# SystemdCgroupを有効にする
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
```

16.2.4 kubeadm、kubelet、kubectl のインストール

```
# Kubernetes公式リポジトリの追加（Ubuntu）
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/
core:/stable:/v1.31/deb/ /' | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

16.2.5 コントロールプレーンの初期化

```
# コントロールプレーンノードで実行
sudo kubeadm init --pod-network-cidr=10.244.0.0/16

# kubeconfigの設定
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

16.2.6 CNI（ネットワークプラグイン）のインストール

kubeadm で構築したクラスタには CNI プラグインが必要です。

CNI プラグイン	特徴
Flannel	シンプルで導入が容易、学習向け

CNI プラグイン	特徴
Calico	NetworkPolicy サポート、本番実績豊富
Cilium	eBPF ベース、高パフォーマンス、Observability 機能
Weave Net	暗号化通信対応、セットアップが簡単

```
# Calicoのインストール例
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/
manifests/calico.yaml

# Flannelのインストール例
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-
flannel.yml
```

16.2.7 ワーカーノードの追加

`kubeadm init` の出力に含まれるコマンドをワーカーノードで実行します。

```
# ワーカーノードで実行
sudo kubeadm join <コントロールプレーンIP>:6443 \
  --token <トークン> \
  --discovery-token-ca-cert-hash sha256:<ハッシュ>
```

トークンが期限切れの場合は再生成できます。

```
# コントロールプレーンでトークンを再生成
kubeadm token create --print-join-command
```

16.2.8 クラスタのアップグレード

```
# kubeadmのアップグレード
sudo apt-get update
sudo apt-get install -y kubeadm=1.32.0-1.1

# アップグレードプランの確認
sudo kubeadm upgrade plan

# アップグレードの実行（コントロールプレーン）
sudo kubeadm upgrade apply v1.32.0

# kubeletのアップグレード
sudo apt-get install -y kubelet=1.32.0-1.1
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

16.2.9 kubeadm の特徴まとめ

- Kubernetes 公式のクラスタ構築ツール
- 素の Kubernetes を構築でき、カスタマイズの自由度が高い
- CNI、Ingress Controller、監視ツールなどを自分で選択・導入する必要がある
- クラスタの運用・アップグレードは手動で管理する
- CKA（Certified Kubernetes Administrator）試験でも使用される

16.3 Kubespray

16.3.1 Kubespray とは

Kubespray は Ansible ベースの Kubernetes クラスタ構築ツールです。kubeadm を内部で使用しながら、Ansible のプレイブックで複数ノードのセットアップを自動化します。

16.3.2 特徴

- Ansible で構成管理するため、複数ノードの一括セットアップが容易
- kubeadm を内部で使用（kubeadm の自動化ラッパーとも言える）
- オンプレミス、クラウド VM、ベアメタルに対応
- CNI、コンテナランタイム、アドオンの選択をインベントリで設定可能
- HA 構成のクラスタも自動構築できる

16.3.3 基本的な使い方

```
# リポジトリのクローン
git clone https://github.com/kubernetes-sigs/kubespray.git
cd kubespray

# 依存関係のインストール
pip install -r requirements.txt

# インベントリの準備
cp -rfp inventory/sample inventory/my-cluster

# ノード情報を設定（IPアドレスを指定）
declare -a IPS=(10.0.0.1 10.0.0.2 10.0.0.3)
CONFIG_FILE=inventory/my-cluster/hosts.yaml \
  python3 contrib/inventory_builder/inventory.py ${IPS[@]}
```

16.3.4 インベントリのカスタマイズ

```
# inventory/my-cluster/group_vars/k8s_cluster/k8s-cluster.yml
kube_version: v1.31.0
kube_network_plugin: calico     # CNIの選択
```

```
container_manager: containerd   # コンテナランタイム
cluster_name: my-cluster
```

16.3.5 クラスタの構築

```
# クラスタを構築
ansible-playbook -i inventory/my-cluster/hosts.yaml \
  --become --become-user=root \
  cluster.yml

# ノードの追加
ansible-playbook -i inventory/my-cluster/hosts.yaml \
  --become --become-user=root \
  scale.yml

# クラスタのアップグレード
ansible-playbook -i inventory/my-cluster/hosts.yaml \
  --become --become-user=root \
  upgrade-cluster.yml
```

16.3.6 kubeadm との使い分け

- 数台規模なら kubeadm で手動構築でも問題ない
- 10 台以上の大規模構成や繰り返しの構築には Kubespray が効率的
- Infrastructure as Code としてクラスタ構成を Git 管理したい場合にも有用

16.4 Cluster API（CAPI）

16.4.1 Cluster API とは

Cluster API は Kubernetes クラスタ自体を Kubernetes リソースとして宣言的に管理するプロジェクトです。既存の Kubernetes クラスタ（Management Cluster）から、新しいクラスタ（Workload Cluster）のライフサイクルを管理します。

16.4.2 コンセプト

```
Management Cluster（管理クラスタ）
  └── Cluster API コントローラー
        ├── Workload Cluster A
        ├── Workload Cluster B
        └── Workload Cluster C
```

- **Management Cluster**: Cluster API のコントローラーが動作するクラスタ
- **Workload Cluster**: 実際にアプリケーションを動かすクラスタ
- **Infrastructure Provider**: クラスタが動作するインフラ（AWS, Azure, vSphere, Docker 等）

- **Bootstrap Provider**: ノードの初期化方法（kubeadm, Talos, MicroK8s 等）
- **Control Plane Provider**: コントロールプレーンの管理方法

16.4.3 対応インフラプロバイダー

プロバイダー	略称	対応インフラ
CAPA	AWS	EC2, EKS
CAPZ	Azure	Azure VM, AKS
CAPG	GCP	GCE, GKE
CAPV	vSphere	VMware vSphere
CAPH	Hetzner	Hetzner Cloud / ベアメタル
CAPD	Docker	ローカル Docker（開発用）
CAPT	Talos	Talos Linux

16.4.4 clusterctl のインストール

```
# macOS
brew install clusterctl

# Linux
curl -L https://github.com/kubernetes-sigs/cluster-api/releases/latest/download/
clusterctl-linux-amd64 -o clusterctl
chmod +x clusterctl
sudo mv clusterctl /usr/local/bin/
```

16.4.5 Management Cluster の初期化

```
# AWSプロバイダーで初期化する例
export AWS_REGION=ap-northeast-1
export AWS_ACCESS_KEY_ID=<アクセスキー>
export AWS_SECRET_ACCESS_KEY=<シークレットキー>

clusterctl init --infrastructure aws
```

16.4.6 Workload Cluster の作成

```
# クラスタのマニフェストを生成
clusterctl generate cluster my-cluster \
  --infrastructure aws \
  --kubernetes-version v1.31.0 \
  --control-plane-machine-count 3 \
  --worker-machine-count 3 \
  > my-cluster.yaml

# マニフェストを適用してクラスタを作成
```

```
kubectl apply -f my-cluster.yaml

# クラスタの状態を確認
kubectl get clusters
clusterctl describe cluster my-cluster

# Workload Clusterのkubeconfigを取得
clusterctl get kubeconfig my-cluster > my-cluster-kubeconfig
```

16.4.7 Cluster API のユースケース

- 複数の Kubernetes クラスタを統一的に管理したい
- クラスタの作成・削除・アップグレードを自動化したい
- GitOps でクラスタのライフサイクルを管理したい
- マルチクラウド・ハイブリッドクラウド環境を構築したい

16.5 k3s（本番環境での使用）

第 2 章では k3s をローカル開発ツールとして紹介しましたが、k3s は本番環境でも広く使われています。

16.5.1 k3s の本番向け構成

16.5.1.1 高可用性（HA）クラスタ

k3s はサーバー（コントロールプレーン）を複数台構成にすることで高可用性を実現できます。

```
# 1台目のサーバー（埋め込みetcdで初期化）
curl -sfL https://get.k3s.io | sh -s - server \
  --cluster-init \
  --tls-san <ロードバランサーIP>

# 2台目以降のサーバー
curl -sfL https://get.k3s.io | sh -s - server \
  --server https://<1台目のIP>:6443 \
  --token <トークン> \
  --tls-san <ロードバランサーIP>

# ワーカーノードの追加
curl -sfL https://get.k3s.io | sh -s - agent \
  --server https://<ロードバランサーIP>:6443 \
  --token <トークン>
```

16.5.1.2 外部データベースの使用

大規模環境では etcd の代わりに MySQL、PostgreSQL、etcd クラスタを使用できます。

```
curl -sfL https://get.k3s.io | sh -s - server \
  --datastore-endpoint="postgres://user:pass@db-host:5432/k3s"
```

16.5.2 kubeadm との比較

観点	kubeadm	k3s
バイナリサイズ	大（複数コンポーネント）	約 70MB（単一バイナリ）
セットアップ	手順が多い	ワンライナーで完了
コンポーネント	個別にインストール	Traefik、CoreDNS 等が組み込み
データストア	etcd	SQLite / 外部 DB / 埋め込み etcd
リソース消費	高	低
カスタマイズ性	高	中
ARM 対応	限定的	完全対応
CNCF 認定	対象（バニラ K8s）	認定済み

16.6 k0s

16.6.1 k0s とは

k0s は Mirantis 社が開発したゼロフリクション（摩擦ゼロ）を目指す軽量 Kubernetes ディストリビューションです。k3s と同様に単一バイナリで動作しますが、設計思想に違いがあります。

16.6.2 k0s の特徴

- **ゼロ依存**: ホスト OS に追加パッケージが不要（コンテナランタイムも内蔵）
- **ゼロフリクション**: 標準的な Kubernetes API を 100% 維持（独自変更なし）
- **ゼロコスト**: 完全オープンソース（Apache 2.0 ライセンス）
- **k0sctl**: SSH ベースのクラスタ管理ツールで複数ノードを一括管理
- **自動アップグレード**: Autopilot 機能によるクラスタの自動更新

16.6.3 k3s との違い

観点	k3s	k0s
開発元	SUSE / Rancher	Mirantis
組み込みコンポーネント	Traefik, ServiceLB, Local Path Provisioner	最小限（自分で選択）
データストア	SQLite / etcd / 外部 DB	SQLite / etcd / 外部 DB
コントロールプレーン分離	サーバーもワークロード実行可	コントローラーはワークロード非実行
クラスタ管理	CLI	k0sctl（SSH 経由の宣言的管理）
自動アップグレード	system-upgrade-controller	Autopilot（組み込み）

16.6.4 インストールと起動

```
# k0sのインストール
curl -sSLf https://get.k0s.sh | sh

# コントローラーとして起動（シングルノード）
sudo k0s install controller --single
sudo k0s start

# kubeconfigの取得
sudo k0s kubeconfig admin > ~/.kube/config
```

16.6.5 k0sctl によるマルチノード構成

k0sctl は SSH 経由で複数ノードのクラスタを宣言的に管理するツールです。

```
# k0sctlのインストール
brew install k0sproject/tap/k0sctl
```

```
# k0sctl.yaml
apiVersion: k0sctl.k0sproject.io/v1beta1
kind: Cluster
metadata:
  name: my-cluster
spec:
  hosts:
    - role: controller
      ssh:
        address: 10.0.0.1
        user: ubuntu
        keyPath: ~/.ssh/id_rsa
    - role: worker
      ssh:
        address: 10.0.0.2
        user: ubuntu
        keyPath: ~/.ssh/id_rsa
    - role: worker
      ssh:
        address: 10.0.0.3
        user: ubuntu
        keyPath: ~/.ssh/id_rsa
  k0s:
    version: "1.31.0+k0s.0"
```

```
# クラスタの構築
k0sctl apply --config k0sctl.yaml

# kubeconfigの取得
```

```
k0sctl kubeconfig --config k0sctl.yaml > ~/.kube/config

# クラスタのリセット
k0sctl reset --config k0sctl.yaml
```

16.7 Talos Linux

16.7.1 Talos Linux とは

Talos Linux は Kubernetes 専用に設計されたイミュータブル（不変）な Linuxディストリビューションです。SSH アクセスやシェルが存在せず、すべての操作を API を通じて行います。セキュリティと運用の自動化に特化しています。

16.7.2 Talos の特徴

- **イミュータブル OS**: ルートファイルシステムは読み取り専用で、改ざんを防止
- **SSH なし**: シェルアクセスが存在しないため、攻撃面が極めて小さい
- **API 駆動**: すべての設定変更を gRPC API で実施
- **宣言的設定**: YAML の設定ファイルで OS・Kubernetes の状態を定義
- **自動アップデート**: OS と Kubernetes を一体でアップグレード
- **最小構成**: 不要なパッケージやサービスが一切入っていない
- **セキュアブート対応**: UEFI セキュアブートをサポート
- **ディスク暗号化**: LUKS2 によるデータパーティションの暗号化をサポート

16.7.3 talosctl のインストール

```
# macOS
brew install siderolabs/tap/talosctl

# Linux
curl -sL https://talos.dev/install | sh
```

16.7.4 クラスタの構成ファイル生成

```
# 設定ファイルの生成
talosctl gen config my-cluster https://<コントロールプレーンIP>:6443

# 以下のファイルが生成される
# controlplane.yaml  - コントロールプレーンの設定
# worker.yaml        - ワーカーノードの設定
# talosconfig        - talosctlの設定
```

16.7.5 ノードへの設定適用

```
# コントロールプレーンに設定を適用
talosctl apply-config --insecure \
  --nodes <ノードIP> \
  --file controlplane.yaml

# ワーカーノードに設定を適用
talosctl apply-config --insecure \
  --nodes <ノードIP> \
  --file worker.yaml
```

16.7.6 クラスタのブートストラップ

```
# talosctlの設定
export TALOSCONFIG="talosconfig"
talosctl config endpoint <コントロールプレーンIP>
talosctl config node <コントロールプレーンIP>

# クラスタのブートストラップ
talosctl bootstrap

# kubeconfigの取得
talosctl kubeconfig
```

16.7.7 設定のカスタマイズ

Talos の設定は YAML で宣言的に管理します。

```
# controlplane.yaml（抜粋）
machine:
  type: controlplane
  network:
    hostname: cp-1
    interfaces:
      - interface: eth0
        dhcp: true
  install:
    disk: /dev/sda
    image: ghcr.io/siderolabs/installer:v1.8.0
  features:
    kubernetesTalosAPIAccess:
      enabled: true
      allowedRoles:
        - os:reader
      allowedKubernetesNamespaces:
        - kube-system
cluster:
```

```yaml
  clusterName: my-cluster
  controlPlane:
    endpoint: https://10.0.0.10:6443
  network:
    cni:
      name: flannel
  proxy:
    disabled: true   # Cilium等を使う場合はkube-proxyを無効化
```

16.7.8 クラスタの管理

```bash
# ノードの状態確認
talosctl get members

# サービスの状態確認
talosctl services

# ログの確認
talosctl logs kubelet

# OS のアップグレード
talosctl upgrade --image ghcr.io/siderolabs/installer:v1.9.0

# Kubernetesのアップグレード
talosctl upgrade-k8s --to 1.32.0

# 設定変更の適用（差分のみ適用される）
talosctl apply-config --nodes <ノードIP> --file controlplane.yaml

# ダッシュボード（ターミナルUI）
talosctl dashboard --nodes <ノードIP>
```

16.7.9 Talos のローカル開発環境

Docker 上で Talos クラスタを試すこともできます。

```bash
# Docker上で3ノードクラスタを作成
talosctl cluster create --name my-cluster \
  --controlplanes 1 --workers 2

# 削除
talosctl cluster destroy --name my-cluster
```

16.7.10 Talos Factory / Image Factory

Talos Image Factory を使うと、カスタムのシステム拡張（iscsi、NVIDIA GPU ドライバ、ZFS など）を含むインストールイメージを生成できます。

```
# schematiを使ってカスタムイメージを作成
cat <<EOF > extensions.yaml
customization:
  systemExtensions:
    officialExtensions:
      - siderolabs/iscsi-tools
      - siderolabs/util-linux-tools
EOF
```

16.7.11 Cluster API との連携

Talos は Cluster API のブートストラッププロバイダー（CABPT）とコントロールプレーンプロバイダー（CACPPT）を提供しており、Cluster API を使って Talos クラスタを宣言的に管理できます。

16.8 RKE2 / Rancher

16.8.1 RKE2 とは

RKE2（Rancher Kubernetes Engine 2）は SUSE/Rancher 社が開発するセキュリティ重視の Kubernetes ディストリビューションです。CIS（Center for Internet Security）ベンチマークにデフォルトで準拠しており、米国政府機関でも採用されています。

16.8.2 RKE2 の特徴

- **CIS ハードニング**: デフォルトで CIS Kubernetes ベンチマークに準拠
- **FIPS 140-2 対応**: 暗号化モジュールの政府認証に対応
- **containerd 組み込み**: Docker への依存なし
- **etcd 組み込み**: 外部 etcd 不要
- **NGINX Ingress Controller**: 組み込み済み（k3s の Traefik とは異なる）
- **Canal CNI**: Flannel + Calico の組み合わせをデフォルトで使用

16.8.3 インストール

```
# サーバー（コントロールプレーン）のインストール
curl -sfL https://get.rke2.io | sh -
sudo systemctl enable rke2-server
sudo systemctl start rke2-server

# kubeconfigの設定
mkdir -p ~/.kube
sudo cp /etc/rancher/rke2/rke2.yaml ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config

# トークンの確認
sudo cat /var/lib/rancher/rke2/server/node-token
```

```
# エージェント（ワーカーノード）のインストール
curl -sfL https://get.rke2.io | INSTALL_RKE2_TYPE="agent" sh -

# サーバー情報の設定
sudo mkdir -p /etc/rancher/rke2
cat <<EOF | sudo tee /etc/rancher/rke2/config.yaml
server: https://<サーバーIP>:9345
token: <トークン>
EOF

sudo systemctl enable rke2-agent
sudo systemctl start rke2-agent
```

16.8.4 RKE2 の設定

```
# /etc/rancher/rke2/config.yaml（サーバー）
write-kubeconfig-mode: "0644"
tls-san:
  - my-cluster.example.com
  - 10.0.0.10
cni: cilium            # CNIの変更（デフォルトはcanal）
disable:
  - rke2-ingress-nginx  # 組み込みIngressを無効化する場合
```

16.8.5 Rancher

Rancher は Kubernetes クラスタを統合管理するための Web UI プラットフォームです。RKE2 で構築したクラスタだけでなく、EKS、GKE、AKS、k3s など様々なクラスタを一元管理できます。

```
# Helmでインストール
helm repo add rancher-latest https://releases.rancher.com/server-charts/latest
helm repo update

kubectl create namespace cattle-system

helm install rancher rancher-latest/rancher \
  --namespace cattle-system \
  --set hostname=rancher.example.com \
  --set replicas=3
```

Rancher の主な機能:

- **マルチクラスタ管理**: 複数の K8s クラスタを WebUI で一元管理
- **ユーザー管理**: LDAP、AD、GitHub、OIDC 連携
- **アプリケーションカタログ**: Helm チャートの管理とデプロイ
- **クラスタプロビジョニング**: RKE2/k3s クラスタの作成を GUI で実行
- **監視・アラート**: Prometheus/Grafana の統合管理

16.8.6 k3s / RKE2 / kubeadm の比較

観点	kubeadm	k3s	RKE2
開発元	Kubernetes 公式	SUSE / Rancher	SUSE / Rancher
想定用途	汎用	エッジ・軽量	セキュリティ重視の本番
CIS ハードニング	手動	手動	デフォルト準拠
FIPS 対応	なし	なし	あり
リソース消費	高	低	中
組み込み Ingress	なし	Traefik	NGINX
データストア	etcd	SQLite/etcd/外部 DB	etcd
管理 UI	なし	なし	Rancher 連携

16.9 OpenShift

16.9.1 OpenShift とは

OpenShift は Red Hat が開発・提供するエンタープライズ向け Kubernetes プラットフォームです。Kubernetes をベースに、CI/CD、モニタリング、ログ、レジストリ、開発者ポータルなどが統合されています。

16.9.2 OpenShift のエディション

エディション	説明	費用
OKD	コミュニティ版（OpenShift のアップストリーム）	無料
OpenShift Container Platform	Red Hat のサブスクリプション付き商用版	有料
OpenShift Dedicated	マネージドサービス（AWS / GCP 上）	有料
ROSA	AWS 上のマネージド OpenShift	有料
ARO	Azure 上のマネージド OpenShift	有料

16.9.3 標準 Kubernetes との主な違い

機能領域	標準 Kubernetes	OpenShift
インストール	kubeadm 等で手動構築	openshift-install で自動構築
Web UI	Dashboard（別途導入）	高機能な Web Console 標準搭載
CI/CD	別途導入が必要	OpenShift Pipelines（Tekton）統合
イメージレジストリ	別途導入が必要	内蔵レジストリ
ルーティング	Ingress Controller 別途導入	Route/HAProxy 標準搭載

機能領域	標準 Kubernetes	OpenShift
セキュリティ	RBAC + PodSecurityStandards	SCC（Security Context Constraints）
モニタリング	別途導入が必要	Prometheus / Grafana 統合済み
ログ	別途導入が必要	OpenShift Logging（Loki）統合
GitOps	別途導入が必要	OpenShift GitOps（ArgoCD）統合
サービスメッシュ	別途導入が必要	OpenShift Service Mesh（Istio）統合
OS	任意の Linux	RHCOS / RHEL（イミュータブル OS）

16.9.4 oc コマンド

OpenShift では `kubectl` に加えて `oc` コマンドが使用できます。`oc` は `kubectl` の上位互換で、OpenShift 固有の機能にもアクセスできます。

```
# ログイン
oc login https://api.my-cluster.example.com:6443

# プロジェクト（Namespaceに相当）の作成
oc new-project my-project

# アプリケーションのデプロイ（ソースから自動ビルド）
oc new-app https://github.com/your-org/your-app.git

# ビルドの確認
oc get builds

# Routeの作成（外部公開）
oc expose service my-app

# Routeの確認
oc get routes
```

16.9.5 Route

OpenShift には Ingress の代わりに Route というリソースがあります（Ingress も使用可能）。

```yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: my-app
spec:
  host: my-app.apps.my-cluster.example.com
  to:
    kind: Service
```

```
    name: my-app
  port:
    targetPort: 8080
  tls:
    termination: edge
```

16.9.6 Security Context Constraints（SCC）

OpenShift はデフォルトで Pod のセキュリティを厳しく制限しています。

```
# 利用可能なSCCを確認
oc get scc

# SCCの詳細を表示
oc describe scc restricted-v2
```

主な SCC:

- **restricted-v2**: デフォルト。root 実行不可、特権コンテナ不可
- **anyuid**: 任意の UID での実行を許可
- **privileged**: すべての制限を解除（管理用途のみ）

16.9.7 ローカル開発環境（OKD / CRC）

OpenShift をローカルで試すには、CodeReady Containers（CRC）を使用します。

```
# CRCのインストール（Red Hatアカウントが必要）
crc setup
crc start

# ocコマンドの設定
eval $(crc oc-env)
oc login -u developer -p developer https://api.crc.testing:6443
```

16.10 VMware Tanzu Kubernetes Grid

16.10.1 Tanzu とは

VMware Tanzu Kubernetes Grid（TKG）は Broadcom（旧 VMware）が提供するエンタープライズ向け Kubernetes プラットフォームです。vSphere 環境との統合が強力で、VMware の既存インフラを活用した Kubernetes 運用ができます。

16.10.2 Tanzu の主な製品

製品	説明
TKG（Tanzu Kubernetes Grid）	マルチクラスタ管理、vSphere/AWS/Azure 対応
TKGs（vSphere with Tanzu）	vSphere に統合された Kubernetes（Supervisor Cluster）
TCE（Tanzu Community Edition）	コミュニティ版（現在はアーカイブ）

16.10.3 特徴

- vSphere 環境で VM と同じ管理フレームワークで Kubernetes を運用
- Cluster API を内部で使用しクラスタをプロビジョニング
- NSX-T との連携による高度なネットワーキング
- Harbor（コンテナレジストリ）との統合

16.11 コンテナ最適化 OS

Kubernetes ノード用に最適化された軽量 OS が存在します。汎用 Linux ディストリビューション（Ubuntu、RHEL 等）の代わりに使用することで、セキュリティの向上と運用の簡素化が実現できます。

16.11.1 Bottlerocket

Bottlerocket は AWS が開発したコンテナ実行専用の Linux ディストリビューションです。

- **AWS 製**: EKS、ECS との統合が最適化されている
- **イミュータブル**: ルートファイルシステムは読み取り専用
- **API 駆動**: 設定変更は API または user-data 経由
- **A/B アップデート**: パーティションの切り替えによる安全な OS アップデート
- **最小構成**: パッケージマネージャー、シェルアクセスなし（管理用コンテナ経由で可能）

```
# EKSでBottlerocketノードグループを作成
eksctl create nodegroup \
  --cluster my-cluster \
  --node-ami-family Bottlerocket \
  --name bottlerocket-ng
```

16.11.2 Flatcar Container Linux

Flatcar Container Linux は CoreOS Container Linux（現在は Red Hat 傘下）の後継として、Kinvolk が開発を引き継いだコンテナ最適化 OS です。

- **CoreOS の後継**: Container Linux の互換性を維持
- **自動アップデート**: Nebraska アップデートサーバーによる A/B アップデート
- **Ignition**: 初回起動時の宣言的設定（cloud-init の代替）

- **クラウド非依存**: AWS、GCP、Azure、ベアメタルすべてに対応
- **sysext**: systemd-sysext によるカスタムソフトウェアの追加

```
# Ignition設定の例（Butane形式で記述してIgnitionに変換）
variant: flatcar
version: 1.0.0
storage:
  files:
    - path: /etc/hostname
      contents:
        inline: worker-1
systemd:
  units:
    - name: containerd.service
      enabled: true
```

16.11.3 コンテナ最適化 OS の比較

観点	Talos Linux	Bottlerocket	Flatcar
開発元	Sidero Labs	AWS	Kinvolk / Microsoft
用途	K8s ノード OS + 構築	K8s ノード OS	K8s ノード OS
K8s 構築機能	あり（OS 自体が K8s を管理）	なし（EKS 等と併用）	なし（kubeadm 等と併用）
SSH	なし	管理コンテナ経由	あり
クラウド対応	全般	AWS 中心	全般
設定方式	YAML（talosctl）	TOML（API）	Ignition

16.12 マネージド Kubernetes サービス

クラウドプロバイダーが提供するマネージド Kubernetes サービスでは、コントロールプレーンの運用をプロバイダーが担当します。

サービス	プロバイダー	特徴
EKS	AWS	IAM 統合、Fargate によるサーバーレスノード、Bottlerocket 対応
GKE	Google Cloud	Autopilot モード、高度なネットワーキング、Kubernetes の本家
AKS	Azure	Azure AD 統合、Azure Arc 対応、Windows ノード対応
LKE	Linode / Akamai	シンプルで低コスト、小〜中規模向け
DOKS	DigitalOcean	シンプルな UI、小規模プロジェクト向け
OKE	Oracle Cloud	Always Free 枠あり、Oracle DB 統合

マネージドサービスはコントロールプレーンの運用が不要で、ノードのスケーリングやアップグレードも簡単です。本番環境ではマネージドサービスの利用が広く推奨されています。

16.12.1 CLI ツール

各マネージドサービスにはクラスタ管理用の CLI があります。

```
# EKS（AWS）
eksctl create cluster --name my-cluster --region ap-northeast-1
aws eks update-kubeconfig --name my-cluster --region ap-northeast-1

# GKE（Google Cloud）
gcloud container clusters create my-cluster --zone asia-northeast1-a
gcloud container clusters get-credentials my-cluster --zone asia-northeast1-a

# AKS（Azure）
az aks create --resource-group myRG --name my-cluster
az aks get-credentials --resource-group myRG --name my-cluster
```

16.13 ディストリビューションの選び方

要件	推奨
Kubernetes の内部を学びたい	kubeadm（構築プロセスを理解できる）
大規模オンプレミスの自動構築	Kubespray（Ansible で自動化）
クラスタ自体を K8s で管理したい	Cluster API（宣言的なマルチクラスタ管理）
小規模・エッジ・IoT	k3s（軽量、ARM 対応、簡単セットアップ）
標準準拠の軽量 K8s	k0s（ゼロ依存、標準 API 100% 維持）
セキュリティ最優先	Talos Linux（イミュータブル OS、SSH 不要）
政府・金融機関向け	RKE2（CIS デフォルト準拠、FIPS 対応）
エンタープライズ（Red Hat 統合）	OpenShift（統合ツール群、サポート付き）
エンタープライズ（VMware 統合）	Tanzu（vSphere 連携、NSX-T 統合）
マルチクラスタの Web 管理	Rancher（あらゆる K8s クラスタを一元管理）
クラウドで手軽に本番運用	EKS / GKE / AKS（マネージドで運用負荷が低い）
クラウド非依存の本番環境	Talos + Cilium または RKE2